BỘ GIÁO DỤC VÀ ĐÀO TẠO

TRƯỜNG ĐẠI HỌC SƯ PHẠM KỸ THUẬT HƯNG YÊN



BÀI TẬP LỚN

THIẾT KẾ HỆ THỐNG IOT

HỆ THỐNG IOT GIÁM SÁT VÀ CẢNH BÁO ĐIỀU KIỆN BẢO QUẢN THỰC PHẨM

NGÀNH: CÔNG NGHỆ THÔNG TIN

SINH VIÊN: **LÊ NGUYỄN MẠNH HÙNG - 10123152**

LÊ QUANG HUÂN – 10123150

LỚP: **10123O.1**

NGƯỜI HƯỚNG DẪN: **CHU BÁ THÀNH**

HƯNG YÊN – 2026

NHẬN XÉT

Nhận xét của giáo viên hướng dẫn

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

GIÁO VIÊN HƯỚNG DẪN

LỜI CAM ĐOAN

Chúng em xin cam đoan bài tập lớn môn Thiết kế hệ thống IoT có tên là "*Hệ thống IoT giám sát và cảnh báo điều kiện bảo quản thực phẩm*" là sản phẩm của chúng em.

Những phần sử dụng tài liệu tham khảo trong bài tập lớn đã được nêu rõ trong phần tài liệu tham khảo. Các kết quả trình bày trong bài tập lớn hoàn toàn là kết quả do chúng em thực hiện.

Nếu vi phạm lời cam đoan này, chúng em xin chịu hoàn toàn trách nhiệm trước khoa và nhà trường.

Hưng Yên, ngày ... tháng ... năm 2026

Sinh viên	*Sinh viên*
(Ký và ghi rõ họ tên)	*(Ký và ghi rõ họ tên)*
……	……

LỜI CẢM ƠN

Để có thể hoàn thành bài tập này, lời đầu tiên chúng em xin phép gửi lời cảm ơn tới bộ môn Hệ thống thông tin, Khoa Công nghệ thông tin – Trường Đại học Sư phạm Kỹ thuật Hưng Yên đã tạo điều kiện thuận lợi cho chúng em thực hiện bài tập lớn môn học này.

Đặc biệt chúng em xin chân thành cảm ơn thầy Chu Bá Thành đã rất tận tình hướng dẫn, chỉ bảo chúng em trong suốt thời gian thực hiện bài tập vừa qua.

Chúng em cũng xin chân thành cảm ơn tất cả các Thầy, các Cô trong Trường đã tận tình giảng dạy, trang bị cho chúng em những kiến thức cần thiết, quý báu để giúp chúng em thực hiện được bài tập này.

Mặc dù chúng em đã có cố gắng, nhưng với trình độ còn hạn chế, trong quá trình thực hiện đề tài không tránh khỏi những thiếu sót. Chúng em hy vọng sẽ nhận được những ý kiến nhận xét, góp ý của các Thầy cô về những kết quả triển khai trong bài tập lớn.

Chúng em xin trân trọng cảm ơn!

MỤC LỤC

LỜI CAM ĐOAN 3
LỜI CẢM ƠN 4
MỤC LỤC 5
DANH MỤC HÌNH ẢNH 9
CHƯƠNG 1. TỔNG QUAN VỀ ĐỀ TÀI 10
1.1. Lý do chọn đề tài 10
1.2. Mục tiêu đề tài 11
1.2.1. Mục tiêu tổng quát 11
1.2.2. Mục tiêu cụ thể 11
1.3. Đối tượng và phạm vi nghiên cứu 12
1.3.1. Đối tượng nghiên cứu 12
1.3.2. Phạm vi nghiên cứu 13
1.4. Phương pháp nghiên cứu 14
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT 15
2.1. Tổng quan về công nghệ Internet vạn vật 15
2.1.1. Khái niệm IoT 15
2.1.2. Kiến trúc cơ bản của hệ thống IoT 15
2.1.3. Vai trò của IoT trong giám sát kho hàng 16
2.2. Các thiết bị phần cứng sử dụng trong hệ thống 16
2.2.1. Vi điều khiển ESP32 16
2.2.2. Cảm biến nhiệt độ và độ ẩm DHT22 17
2.2.3. Cảm biến gia tốc MPU6050 18
2.2.4. Màn hình LCD giao tiếp I2C 18
2.2.5. Module relay và các thiết bị đầu ra 19

2.3. Công nghệ truyền thông và trao đổi dữ liệu 20
2.3.1. Giao thức MQTT 20
2.3.2. Định dạng dữ liệu JSON 21
2.4. Công nghệ xây dựng hệ thống phần mềm 22
2.4.1. Node.js và Express.js 22
2.4.2. WebSocket 22
2.4.3. Cơ sở dữ liệu MySQL 23
2.5. Nguyên lý giám sát và điều khiển tự động 24
2.5.1. Nguyên lý giám sát theo ngưỡng 24
2.5.2. Chế độ điều khiển tự động 25
2.5.3. Chế độ điều khiển thủ công 25
CHƯƠNG 3. PHÂN TÍCH VÀ THIẾT HỆ THỐNG 26
3.1. Phân tích yêu cầu hệ thống 26
3.1.1. Yêu cầu chức năng 26
3.1.2. Yêu cầu phi chức năng 27
3.1.3. Các thông số và điều kiện giám sát 28
3.2. Thiết kế kiến trúc tổng thể 28
3.2.1. Mô hình kiến trúc hệ thống 28
3.2.2. Nguyên lý trao đổi dữ liệu giữa các thành phần 29
3.3. Thiết kế phần cứng 30
3.3.1. Sơ đồ khối phần cứng 30
3.3.2. Thiết kế nối cảm biến và thiết bị 31
3.4. Thiết kế phần mềm 32
3.4.1. Thiết kế chương trình trên ESP32 32
3.4.2. Thiết kế backend 32

3.4.3. Thiết kế cơ sở dữ liệu 33
3.4.4. Thiết kế giao diện dashboard 34
3.5. Thiết kế thuật toán điều khiển 35
3.5.1. Thuật toán giám sát nhiệt độ và độ ẩm 35
3.5.2. Thuật toán cảnh báo rung động 36
3.5.3. Thuật toán điều khiển theo chế độ vận hành 36
3.6. Thiết kế quy trình kiểm thử hệ thống 38
CHƯƠNG 4. XÂY DỰNG VÀ TRIỂN KHAI HỆ THỐNG 39
4.1. Xây dựng phần cứng hệ thống 39
4.1.1. Lắp ráp các thiết bị phần cứng 39
4.1.2. Kết nối các cảm biến và các thiết bị ngoại vi 39
4.2. Xây dựng chương trình cho ESP32 40
4.2.1. Thiết lập môi trường lập 40
4.2.2. Thu thập dữ liệu nhiệt độ và độ ẩm 41
4.2.3. Thu thập và xử lý dữ liệu rung 42
4.2.4. Điều khiển theo chế độ hoạt động 42
4.3. Xây dựng hệ thống truyền thông MQTT 43
4.3.1. Xây dựng các kênh truyền dữ liệu 43
4.3.2. Kiểm tra quá trình truyền nhận dữ liệu 44
4.4. Xây dựng backend xử lý dữ liệu 44
4.4.1. Thiết lập môi trường backend 44
4.4.2. Tiếp nhận và xử lý dữ liệu MQTT 45
4.4.3. Xây dựng API và truyền dữ liệu thời gian thực 45
4.5. Xây dựng cơ sở dữ liệu MySQL 46
4.5.1. Thiết lập cơ sở dữ liệu 46

4.5.2. Lưu trữ lịch sử dữ liệu cảm biến 46
4.5.3. Truy vấn dữ liệu lịch sử 47
4.6. Xây dựng giao diện giám sát và điều khiển 48
4.6.1. Thiết lập môi trường giao diện 48
4.6.2. Xây dựng màn hình Dashboard 48
4.6.3. Xây dựng chức năng điều khiển thiết bị 49
4.6.4. Xây dựng màn hình lịch sử và cảnh báo 50
4.7. Kiểm thử và đánh giá kết quả 51
4.7.1. Mục tiêu kiểm thử 51
4.7.2. Kết quả kiểm thử chức năng 52
4.7.3. Kiểm thử chức năng cảnh báo môi trường 53
4.7.4. Kiểm thử truyền dữ liệu và lưu trữ 53
4.7.5. Đánh giá ưu điểm và hạn chế 54
5.1. Kết luận 55
5.2. Hạn chế 55
5.3. Hướng phát triển 55
5.4. Tổng kết 56
TÀI LIỆU THAM KHẢO 57

DANH MỤC HÌNH ẢNH

Hình 3.1. Sơ đồ kiến trúc tổng thể của hệ thống IoT giám sát kho hàng..... 29
Hình 3.2. Sơ đồ khối phần cứng của hệ thống. 30
Hình 4.1. Mô hình phần cứng hệ thống IoT giám sát kho hàng. 40
Hình 4.2. Lưu đồ xử lý dữ liệu và điều khiển thiết bị trên ESP32..... 43
Hình 4.3. Kết quả khởi động backend và kết nối với hệ thống..... 46
Hình 4.4. Kết quả truy vấn lịch sử dữ liệu cảm biến từ API backend. 48
Hình 4.5. Giao diện Dashboard giám sát nhiệt độ, độ ẩm và rung động. 49
Hình 4.6. Giao diện lịch sử dữ liệu cảm biến..... 51
Hình 4.7. Giao diện theo dõi cảnh báo của hệ thống. 51

CHƯƠNG 1. TỔNG QUAN VỀ ĐỀ TÀI

1.1. Lý do chọn đề tài

Trong bối cảnh khoa học và công nghệ phát triển mạnh mẽ, việc ứng dụng công nghệ thông tin vào hoạt động quản lý, giám sát và điều khiển đang trở thành xu hướng phổ biến. Đặc biệt, công nghệ Internet vạn vật phép các thiết bị kết nối với nhau thông qua mạng Internet, thu thập dữ liệu từ môi trường và truyền thông tin đến hệ thống xử lý. Nhờ đó, người dùng có thể theo dõi trạng thái của các đối tượng cần quản lý, phát hiện những thay đổi bất thường và đưa ra biện pháp xử lý kịp thời.

Kho hàng là nơi lưu trữ và bảo quản hàng hóa trước khi được vận chuyển hoặc phân phối đến các đơn vị sử dụng. Đối với kho lưu trữ thực phẩm và nông sản, điều kiện môi trường có ảnh hưởng đáng kể đến chất lượng và thời gian bảo quản sản phẩm. Nhiệt độ và độ ẩm không phù hợp có thể làm thay đổi chất lượng hàng hóa, trong khi các tác động rung động bất thường có thể là dấu hiệu cần được kiểm tra nhằm bảo đảm điều kiện vận hành và an toàn của khu vực lưu trữ. Vì vậy, việc theo dõi thường xuyên các thông số môi trường và phát hiện những tình huống bất thường là một yêu cầu cần thiết trong công tác quản lý kho hàng.

Tuy nhiên, phương pháp giám sát thủ công vẫn tồn tại một số hạn chế như phụ thuộc vào con người, khó theo dõi liên tục và có thể chậm phát hiện khi các thông số vượt quá giới hạn cho phép. Bên cạnh đó, nếu không có hệ thống lưu trữ dữ liệu, việc xem lại diễn biến của các thông số theo thời gian cũng gặp nhiều khó khăn. Do đó, việc xây dựng một hệ thống có khả năng tự động thu thập dữ liệu, hiển thị thông tin, lưu trữ lịch sử và cảnh báo khi xuất hiện điều kiện bất thường sẽ góp phần nâng cao hiệu quả giám sát.

Xuất phát từ những yêu cầu trên, nhóm lựa chọn đề tài “Hệ thống IoT giám sát và cảnh báo điều kiện bảo quản thực phẩm” nhằm nghiên cứu và xây dựng một mô hình giám sát ứng dụng công nghệ IoT. Hệ thống sử dụng vi điều khiển ESP32 kết hợp với cảm biến DHT22 để đo nhiệt độ, độ ẩm và cảm biến MPU6050 để thu thập dữ liệu gia tốc, phục vụ việc theo dõi rung động. Dữ liệu được truyền

thông qua giao thức MQTT đến hệ thống xử lý, sau đó hiển thị trên giao diện dashboard và lưu trữ trong cơ sở dữ liệu MySQL để phục vụ việc theo dõi lịch sử.

Thông qua đề tài, nhóm mong muốn vận dụng các kiến thức về vi điều khiển, cảm biến, truyền thông mạng, lập trình web và cơ sở dữ liệu để xây dựng một mô hình IoT có tính ứng dụng thực tiễn, đồng thời tích lũy kinh nghiệm trong quá trình thiết kế, tích hợp và triển khai một hệ thống giám sát hoàn chỉnh.

1.2. Mục tiêu đề tài

1.2.1. Mục tiêu tổng quát

Thiết kế và xây dựng mô hình hệ thống IoT giám sát kho hàng dành cho môi trường lưu trữ thực phẩm và nông sản, có khả năng thu thập dữ liệu nhiệt độ, độ ẩm và rung động; truyền, hiển thị và lưu trữ dữ liệu; phát hiện các điều kiện bất thường; đồng thời hỗ trợ cảnh báo và điều khiển thiết bị theo chế độ tự động hoặc thủ công.

1.2.2. Mục tiêu cụ thể

Để đạt được mục tiêu tổng quát, đề tài tập trung thực hiện các mục tiêu cụ thể sau:

- Xây dựng mô hình phần cứng: Sử dụng ESP32 làm bộ điều khiển trung tâm, kết nối với cảm biến DHT22, MPU6050, màn hình LCD 16x2 I2C, module relay, quạt thông gió, đèn báo và còi cảnh báo.
- Thu thập dữ liệu cảm biến: Đọc và xử lý các thông số nhiệt độ, độ ẩm và dữ liệu gia tốc phục vụ việc giám sát rung động trong mô hình kho hàng.
- Thiết lập kết nối IoT: Sử dụng giao thức MQTT thông qua nền tảng HiveMQ Cloud để truyền dữ liệu từ ESP32 đến hệ thống xử lý và tiếp nhận lệnh điều khiển từ phía máy chủ.
- Xây dựng hệ thống backend: Phát triển chương trình phía máy chủ để tiếp nhận dữ liệu, xử lý thông tin và cung cấp dữ liệu cho giao diện giám sát.
- Xây dựng cơ sở dữ liệu: Sử dụng MySQL để lưu trữ dữ liệu cảm biến và thông tin cảnh báo, hỗ trợ truy xuất lịch sử phục vụ quá trình theo dõi và đánh giá.

- Phát triển giao diện dashboard: Xây dựng giao diện web cho phép người dùng quan sát các thông số hiện tại, theo dõi biểu đồ, xem lịch sử dữ liệu và kiểm tra trạng thái hoạt động của hệ thống.
- Xây dựng chức năng cảnh báo: Thiết lập các ngưỡng giám sát để phát hiện khi nhiệt độ, độ ẩm hoặc mức rung động vượt giới hạn đã cấu hình.
- Triển khai chức năng điều khiển: Cho phép hệ thống tự động kích hoạt quạt, đèn báo hoặc còi cảnh báo theo điều kiện đã thiết lập; đồng thời hỗ trợ người dùng điều khiển các thiết bị thông qua chế độ thủ công.
- Kiểm thử và đánh giá: Kiểm tra hoạt động của các thành phần phần cứng và phần mềm, đánh giá khả năng thu thập, truyền, hiển thị, lưu trữ dữ liệu cũng như phản hồi của hệ thống khi xuất hiện các điều kiện bất thường.

1.3. Đối tượng và phạm vi nghiên cứu

1.3.1. Đối tượng nghiên cứu

Đối tượng nghiên cứu của đề tài là hệ thống giám sát kho hàng ứng dụng công nghệ IoT, bao gồm các thành phần phần cứng, phần mềm và phương thức truyền thông dữ liệu giữa các thành phần.

Các đối tượng nghiên cứu cụ thể gồm:

- Vi điều khiển ESP32: Đóng vai trò là bộ điều khiển trung tâm, thực hiện việc thu thập dữ liệu cảm biến, xử lý các điều kiện giám sát, kết nối mạng và điều khiển thiết bị đầu ra.
- Cảm biến DHT22: Được sử dụng để đo nhiệt độ và độ ẩm môi trường tại khu vực mô phỏng kho hàng.
- Cảm biến MPU6050: Thu thập dữ liệu gia tốc để theo dõi rung động và hỗ trợ phát hiện các tình huống rung bất thường.
- Giao thức MQTT và nền tảng HiveMQ Cloud: Được sử dụng để truyền dữ liệu cảm biến và trao đổi lệnh điều khiển giữa ESP32 với hệ thống phía máy chủ.

- Backend: Thực hiện việc tiếp nhận, xử lý và cung cấp dữ liệu cho giao diện giám sát, đồng thời phối hợp với các thành phần liên quan trong hệ thống.
- Cơ sở dữ liệu MySQL: Lưu trữ dữ liệu cảm biến và thông tin cảnh báo để phục vụ việc truy xuất lịch sử.
- Giao diện dashboard: Cho phép người dùng theo dõi thông số, quan sát biểu đồ, xem lịch sử và điều khiển thiết bị.
- Các thiết bị đầu ra: Bao gồm module relay, quạt thông gió, đèn báo và còi cảnh báo, được sử dụng để thực hiện các chức năng điều khiển và cảnh báo.

1.3.2. Phạm vi nghiên cứu

Đề tài được triển khai trong phạm vi thiết kế và xây dựng mô hình giám sát kho hàng ở quy mô thử nghiệm, hướng đến môi trường lưu trữ thực phẩm và nông sản. Phạm vi nghiên cứu tập trung vào các nội dung sau:

- Thiết kế, lắp ráp và tích hợp các linh kiện điện tử trong mô hình kho hàng.
- Thu thập dữ liệu nhiệt độ, độ ẩm và rung động thông qua các cảm biến được kết nối với ESP32.
- Thiết lập kết nối mạng và truyền dữ liệu giữa thiết bị phần cứng với hệ thống máy chủ thông qua giao thức MQTT.
- Xây dựng backend để tiếp nhận, xử lý và cung cấp dữ liệu cho giao diện giám sát.
- Lưu trữ dữ liệu cảm biến và thông tin cảnh báo trong cơ sở dữ liệu MySQL.
- Xây dựng dashboard để hiển thị thông số, biểu đồ, trạng thái hệ thống và lịch sử dữ liệu.
- Triển khai chức năng cảnh báo khi thông số vượt ngưỡng, kết hợp điều khiển quạt thông gió, đèn báo và còi cảnh báo theo chế độ tự động hoặc thủ công.
- Kiểm tra và đánh giá hoạt động của hệ thống thông qua các tình huống thử nghiệm trên mô hình thực tế.

1.4. Phương pháp nghiên cứu

Để thực hiện đề tài, nhóm kết hợp phương pháp nghiên cứu lý thuyết với phương pháp thiết kế, triển khai và thực nghiệm trên mô hình thực tế.

Phương pháp nghiên cứu tài liệu: Tìm hiểu các tài liệu liên quan đến công nghệ IoT, vi điều khiển ESP32, nguyên lý hoạt động của cảm biến DHT22 và MPU6050, giao thức MQTT, nền tảng HiveMQ Cloud, lập trình backend, cơ sở dữ liệu MySQL và xây dựng giao diện web. Trên cơ sở đó, lựa chọn các công nghệ phù hợp với yêu cầu và điều kiện thực hiện đề tài.

Phương pháp phân tích và thiết kế hệ thống: Xác định các yêu cầu chức năng của hệ thống, phân tích mối quan hệ giữa phần cứng và phần mềm, xây dựng kiến trúc tổng thể và thiết kế luồng dữ liệu. Quá trình này bao gồm việc xác định chức năng của từng linh kiện, phương thức truyền dữ liệu, cách lưu trữ thông tin và cơ chế phối hợp giữa hệ thống giám sát với các thiết bị đầu ra.

Phương pháp xây dựng và triển khai: Tiến hành lắp ráp mô hình phần cứng, lập trình ESP32 để đọc dữ liệu cảm biến và điều khiển thiết bị. Tiếp đó, xây dựng backend, thiết lập kết nối MQTT, tích hợp cơ sở dữ liệu MySQL và phát triển giao diện dashboard. Các thành phần được kết nối và hoàn thiện theo từng giai đoạn để thuận tiện cho việc kiểm tra và phát hiện lỗi.

Phương pháp thực nghiệm: Vận hành hệ thống trong các điều kiện thử nghiệm khác nhau nhằm kiểm tra khả năng thu thập và truyền dữ liệu, hiển thị thông số, lưu trữ lịch sử, phát hiện điều kiện vượt ngưỡng và điều khiển thiết bị. Các chế độ tự động và thủ công được kiểm tra để đánh giá khả năng phản hồi của hệ thống.

Phương pháp kiểm tra và đánh giá: Theo dõi kết quả hoạt động của hệ thống, đối chiếu dữ liệu giữa cảm biến và giao diện giám sát, kiểm tra việc lưu trữ dữ liệu trong cơ sở dữ liệu và đánh giá phản ứng của các thiết bị đầu ra khi điều kiện giám sát thay đổi. Từ kết quả thử nghiệm, xác định những chức năng đã hoàn thành, các hạn chế còn tồn tại và đề xuất hướng cải tiến phù hợp.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

2.1. Tổng quan về công nghệ Internet vạn vật

2.1.1. Khái niệm IoT

Internet vạn vật (Internet of Things – IoT) là công nghệ cho phép các thiết bị vật lý được kết nối với mạng Internet hoặc mạng máy tính để thu thập, trao đổi và xử lý dữ liệu. Các thiết bị IoT thường được tích hợp cảm biến, bộ xử lý và các giao tiếp truyền thông nhằm thực hiện chức năng giám sát, điều khiển hoặc tự động hóa theo những điều kiện được xác định.

Một hệ thống IoT cơ bản thường bao gồm các thành phần chính như thiết bị thu thập dữ liệu, mạng truyền thông, hệ thống xử lý dữ liệu và giao diện tương tác với người dùng. Các thành phần này phối hợp với nhau để hình thành một quy trình khép kín từ thu thập thông tin đến hiển thị kết quả và thực hiện hành động điều khiển.

Trong lĩnh vực quản lý kho hàng, công nghệ IoT có thể được sử dụng để theo dõi các thông số môi trường, phát hiện những điều kiện bất thường và hỗ trợ người quản lý kiểm soát thiết bị. Nhờ khả năng cập nhật dữ liệu thường xuyên và lưu trữ thông tin, hệ thống giúp người dùng theo dõi trạng thái kho hàng thuận tiện hơn so với phương pháp kiểm tra hoàn toàn thủ công.

2.1.2. Kiến trúc cơ bản của hệ thống IoT

Một hệ thống IoT có thể được tổ chức thành các lớp chức năng như sau:

- Lớp cảm nhận: Bao gồm các cảm biến và thiết bị thu thập dữ liệu từ môi trường.
- Lớp truyền thông: Đảm nhiệm việc truyền dữ liệu giữa thiết bị đầu cuối và hệ thống xử lý thông qua các giao thức mạng.
- Lớp xử lý dữ liệu: Tiếp nhận, xử lý và lưu trữ dữ liệu, đồng thời thực hiện các chức năng quản lý và điều khiển.

- Lớp ứng dụng: Cung cấp giao diện để người dùng theo dõi thông tin, xem lịch sử và tương tác với hệ thống.

Trong đề tài này, kiến trúc IoT được triển khai bằng cách sử dụng ESP32 và các cảm biến làm thành phần thu thập dữ liệu. Dữ liệu được truyền qua Wi-Fi bằng giao thức MQTT đến nền tảng HiveMQ Cloud. Backend sử dụng Node.js để tiếp nhận và xử lý dữ liệu, MySQL để lưu trữ thông tin, còn giao diện dashboard được xây dựng bằng React để hiển thị và hỗ trợ điều khiển hệ thống.

2.1.3. Vai trò của IoT trong giám sát kho hàng

Ứng dụng IoT trong giám sát kho hàng mang lại một số lợi ích cơ bản:

- Theo dõi các thông số môi trường thông qua cảm biến.
- Hỗ trợ phát hiện sớm các điều kiện vượt ngưỡng được thiết lập.
- Cung cấp thông tin giám sát trên giao diện tập trung.
- Lưu trữ dữ liệu để phục vụ việc xem lại lịch sử và đánh giá sự thay đổi theo thời gian.
- Hỗ trợ điều khiển thiết bị theo điều kiện tự động hoặc thông qua thao tác của người dùng.

Đối với đề tài này, IoT đóng vai trò kết nối giữa phần cứng tại mô hình kho hàng với hệ thống phần mềm. Qua đó, dữ liệu cảm biến không chỉ được hiển thị tại thiết bị mà còn có thể truyền đến máy chủ, lưu trữ và truy cập thông qua dashboard.

2.2. Các thiết bị phần cứng sử dụng trong hệ thống

2.2.1. Vi điều khiển ESP32

ESP32 là dòng vi điều khiển tích hợp khả năng kết nối Wi-Fi và Bluetooth, được sử dụng phổ biến trong các ứng dụng IoT nhờ khả năng xử lý, giao tiếp với cảm biến và kết nối mạng. Thiết bị hỗ trợ nhiều giao tiếp ngoại vi như GPIO, UART, SPI và I2C, cho phép kết nối với nhiều loại cảm biến và thiết bị đầu ra.

Trong hệ thống, ESP32 đóng vai trò là bộ điều khiển trung tâm, thực hiện các nhiệm vụ chính sau:

- Thu thập dữ liệu từ cảm biến DHT22 và MPU6050.
- Xử lý dữ liệu để xác định trạng thái giám sát theo các ngưỡng đã cấu hình.
- Kết nối mạng Wi-Fi và trao đổi dữ liệu với HiveMQ Cloud thông qua MQTT.
- Nhận lệnh điều khiển từ phía máy chủ.
- Điều khiển module relay để kích hoạt quạt thông gió, đèn báo và còi cảnh báo.
- Gửi dữ liệu và trạng thái hoạt động đến hệ thống giám sát.

Việc sử dụng ESP32 giúp tích hợp chức năng thu thập dữ liệu, xử lý điều kiện và kết nối mạng trên cùng một thiết bị, phù hợp với yêu cầu xây dựng mô hình giám sát kho hàng có chi phí hợp lý.

2.2.2. Cảm biến nhiệt độ và độ ẩm DHT22

DHT22 là cảm biến kỹ thuật số được sử dụng để đo nhiệt độ và độ ẩm tương đối của môi trường. Cảm biến cung cấp dữ liệu thông qua giao tiếp số, giúp vi điều khiển đọc các giá trị đo mà không cần xây dựng mạch chuyển đổi tín hiệu tương tự sang số riêng cho hai đại lượng này.

Hai thông số do DHT22 cung cấp gồm:

- Nhiệt độ: Biểu thị mức nóng hoặc lạnh của môi trường, thường được biểu diễn bằng độ Celsius (°C).
- Độ ẩm tương đối: Biểu thị lượng hơi nước trong không khí so với mức hơi nước bão hòa tại cùng nhiệt độ, được biểu diễn bằng phần trăm (%RH).

Trong hệ thống giám sát kho hàng, DHT22 được kết nối với ESP32 để thu thập dữ liệu môi trường. Sau khi đọc dữ liệu, chương trình so sánh các giá trị với ngưỡng được thiết lập. Nếu nhiệt độ hoặc độ ẩm vượt quá giới hạn, hệ thống có thể kích hoạt chế độ cảnh báo và điều khiển các thiết bị tương ứng.

Ưu điểm của DHT22 là cấu tạo tương đối đơn giản, dễ tích hợp và phù hợp với các mô hình giám sát môi trường. Tuy nhiên, kết quả đo vẫn chịu ảnh hưởng bởi giới hạn kỹ thuật của cảm biến, điều kiện lắp đặt và môi trường sử dụng. Vì vậy, cảm biến cần được bố trí hợp lý để thu được dữ liệu có ý nghĩa đối với khu vực cần giám sát.

2.2.3. Cảm biến gia tốc MPU6050

MPU6050 là cảm biến tích hợp gia tốc kế ba trục và con quay hồi chuyển ba trục. Thiết bị có khả năng thu thập dữ liệu gia tốc theo ba phương X, Y, Z và dữ liệu vận tốc góc quanh các trục tương ứng. MPU6050 hỗ trợ giao tiếp I2C để trao đổi dữ liệu với vi điều khiển.

Trong đề tài, MPU6050 được sử dụng chủ yếu để thu thập dữ liệu gia tốc nhằm theo dõi rung động của mô hình kho hàng. ESP32 đọc dữ liệu từ cảm biến, xử lý và xác định giá trị phục vụ việc so sánh với ngưỡng rung được thiết lập trong chương trình.

Khi mức rung được xác định vượt ngưỡng, hệ thống kích hoạt còi cảnh báo thông qua module relay. Chức năng này giúp minh họa khả năng phát hiện sự thay đổi bất thường về gia tốc và phản hồi của hệ thống.

Tuy nhiên, dữ liệu gia tốc thô có thể bao gồm cả ảnh hưởng của trọng lực, hướng đặt cảm biến và nhiễu đo. Do đó, việc xác định rung động phụ thuộc vào phương pháp xử lý dữ liệu và cách thiết lập ngưỡng. Trong phạm vi đề tài, MPU6050 phục vụ giám sát rung động trên mô hình thử nghiệm, không được xem là thiết bị chuyên dụng để xác định chính xác động đất hoặc đánh giá kết cấu công trình.

2.2.4. Màn hình LCD giao tiếp I2C

LCD 16x2 là màn hình ký tự có khả năng hiển thị hai dòng, mỗi dòng tối đa 16 ký tự. Màn hình thường được sử dụng trong các hệ thống nhúng để hiển thị thông số và trạng thái hoạt động của thiết bị.

Trong đề tài, LCD được kết nối với ESP32 thông qua module giao tiếp I2C. Giao tiếp I2C sử dụng hai đường tín hiệu chính là SDA để truyền dữ liệu và SCL để cung cấp xung đồng hồ. Nhờ đó, số lượng chân kết nối cần sử dụng trên vi điều khiển được giảm so với phương thức điều khiển LCD song song thông thường.

LCD được sử dụng để hiển thị một số thông tin tại mô hình kho hàng, chẳng hạn như nhiệt độ, độ ẩm hoặc trạng thái hoạt động. Chức năng này cho phép người dùng quan sát thông tin trực tiếp tại thiết bị mà không nhất thiết phải truy cập dashboard.

2.2.5. Module relay và các thiết bị đầu ra

Module relay là thiết bị đóng cắt mạch điện được điều khiển bằng tín hiệu điện. Relay cho phép mạch điều khiển công suất thấp điều khiển việc cấp hoặc ngắt nguồn cho tải phù hợp với thông số định mức của module và tiếp điểm.

Trong hệ thống, module relay được sử dụng để điều khiển các thiết bị đầu ra gồm:

- Quạt thông gió: Được kích hoạt khi nhiệt độ hoặc độ ẩm vượt quá ngưỡng được cấu hình, theo logic điều khiển của chương trình.
- Đèn báo: Hỗ trợ biểu thị trạng thái cảnh báo của hệ thống.
- Còi cảnh báo: Phát tín hiệu âm thanh khi hệ thống phát hiện mức rung vượt ngưỡng thiết lập.

ESP32 không trực tiếp cấp dòng điện cho các tải công suất lớn qua chân GPIO mà sử dụng tín hiệu điều khiển module relay. Việc lựa chọn nguồn điện, module relay và cách đấu nối cần bảo đảm phù hợp với điện áp, dòng điện của tải và yêu cầu an toàn điện.

Đối với hệ thống này, chức năng điều khiển được chia thành hai chế độ. Ở chế độ tự động, ESP32 thực hiện điều khiển dựa trên các điều kiện đã lập trình. Ở chế độ thủ công, người dùng gửi lệnh từ giao diện giám sát để điều khiển thiết bị.

2.3. Công nghệ truyền thông và trao đổi dữ liệu

2.3.1. Giao thức MQTT

MQTT (Message Queuing Telemetry Transport) là giao thức truyền thông theo mô hình xuất bản – đăng ký (Publish/Subscribe), được thiết kế để trao đổi thông điệp giữa các thiết bị thông qua một thành phần trung gian gọi là MQTT Broker.

Mô hình MQTT gồm ba thành phần chính:

- Publisher: Thiết bị hoặc ứng dụng gửi thông điệp lên một chủ đề (topic).
- Subscriber: Thiết bị hoặc ứng dụng đăng ký nhận thông điệp từ các chủ đề quan tâm.
- Broker: Thành phần trung gian tiếp nhận thông điệp từ publisher và chuyển tiếp đến các subscriber phù hợp.

Khác với mô hình truyền thông yêu cầu thiết bị gửi dữ liệu trực tiếp đến từng bên nhận, MQTT cho phép bên gửi và bên nhận hoạt động độc lập thông qua broker. Cách tổ chức này phù hợp với hệ thống IoT có nhiều thiết bị và nhu cầu trao đổi dữ liệu thường xuyên.

Trong đề tài, ESP32 gửi dữ liệu cảm biến lên các topic MQTT. Backend đăng ký nhận dữ liệu để xử lý và cập nhật thông tin cho dashboard. Đồng thời, ESP32 đăng ký topic điều khiển để nhận lệnh từ hệ thống phía máy chủ.

Các topic chính được sử dụng gồm:

Topic	**Chức năng**
iot/sensor/data	Truyền dữ liệu cảm biến và các thông tin liên quan từ ESP32
iot/device/control	Truyền lệnh điều khiển thiết bị đến ESP32
iot/device/status	Trao đổi thông tin trạng thái thiết bị

Việc tổ chức dữ liệu theo topic giúp phân biệt thông điệp giám sát, điều khiển và trạng thái, từ đó thuận tiện hơn trong quá trình lập trình và quản lý hệ thống.

MQTT còn hỗ trợ các mức QoS (Quality of Service) khác nhau nhằm xác định cách thức chuyển phát thông điệp. Mức QoS được lựa chọn tùy theo yêu cầu về độ tin cậy, độ trễ và lượng dữ liệu truyền tải của ứng dụng.

2.3.2. Định dạng dữ liệu JSON

JSON (JavaScript Object Notation) là định dạng trao đổi dữ liệu dạng văn bản, tổ chức thông tin dưới dạng các cặp khóa – giá trị và các cấu trúc dữ liệu lồng nhau. JSON được sử dụng phổ biến trong ứng dụng web và hệ thống IoT vì có cấu trúc rõ ràng, dễ đọc và được nhiều ngôn ngữ lập trình hỗ trợ.

Trong hệ thống, dữ liệu cảm biến được đóng gói thành thông điệp JSON trước khi gửi qua MQTT. Các trường dữ liệu có thể bao gồm nhiệt độ, độ ẩm, rung động và trạng thái cảnh báo hoặc điều khiển.

Ví dụ minh họa về cấu trúc dữ liệu:

```
{
  "temperature": 32.0,
  "humidity": 74.0,
  "vibration": 0.6,
  "tempAlert": false,
  "humAlert": false,
  "vibAlert": false
}
```

Các giá trị trong ví dụ chỉ nhằm minh họa cấu trúc JSON, không đại diện cho kết quả đo cố định của hệ thống.

Sau khi nhận thông điệp, backend phân tích nội dung JSON để lấy các trường dữ liệu cần thiết. Thông tin sau đó được sử dụng để cập nhật trạng thái giám sát, truyền dữ liệu đến dashboard và lưu trữ trong cơ sở dữ liệu.

Việc thống nhất tên trường dữ liệu giữa ESP32, backend, cơ sở dữ liệu và dashboard giúp hạn chế sai lệch trong quá trình trao đổi thông tin.

2.4. Công nghệ xây dựng hệ thống phần mềm

2.4.1. Node.js và Express.js

Node.js là môi trường thực thi JavaScript phía máy chủ, cho phép xây dựng các ứng dụng mạng và dịch vụ backend. Node.js hỗ trợ mô hình xử lý bất đồng bộ, phù hợp với các ứng dụng cần tiếp nhận dữ liệu từ nhiều nguồn và thực hiện các thao tác I/O như kết nối mạng hoặc truy cập cơ sở dữ liệu.

Express.js là framework xây dựng ứng dụng web trên nền tảng Node.js. Express cung cấp cơ chế định nghĩa các tuyến xử lý HTTP, xây dựng API và tổ chức các chức năng phía máy chủ.

Trong đề tài, Node.js được sử dụng làm môi trường chạy backend, còn Express.js đảm nhiệm việc xây dựng các API phục vụ giao diện giám sát. Backend có các nhiệm vụ chính:

- Kết nối đến HiveMQ Cloud và đăng ký các topic MQTT cần thiết.
- Tiếp nhận và phân tích dữ liệu cảm biến.
- Ghi dữ liệu và thông tin cảnh báo vào MySQL.
- Cung cấp API để dashboard truy xuất dữ liệu.
- Hỗ trợ trao đổi dữ liệu thời gian thực với giao diện thông qua WebSocket.

Việc sử dụng Node.js và Express.js giúp tập trung các chức năng xử lý dữ liệu và giao tiếp với các thành phần khác trong một chương trình backend.

2.4.2. WebSocket

WebSocket là giao thức cho phép thiết lập một kết nối hai chiều giữa máy khách và máy chủ thông qua một kết nối được duy trì trong thời gian hoạt động.

Sau khi kết nối được thiết lập, hai phía có thể gửi dữ liệu cho nhau mà không cần tạo một yêu cầu HTTP mới cho mỗi lần trao đổi.

Trong hệ thống giám sát kho hàng, WebSocket được sử dụng để truyền dữ liệu từ backend đến dashboard khi có thông tin mới. Nhờ đó, giao diện có thể cập nhật các thông số cảm biến và trạng thái thiết bị mà không cần liên tục tải lại toàn bộ trang.

WebSocket phù hợp với các ứng dụng cần hiển thị dữ liệu thường xuyên như hệ thống giám sát môi trường. Tuy nhiên, việc sử dụng WebSocket không thay thế hoàn toàn HTTP API. Trong hệ thống, HTTP API vẫn có thể được dùng để lấy dữ liệu lịch sử hoặc thực hiện các thao tác truy vấn, trong khi WebSocket phục vụ việc cập nhật thông tin theo thời gian thực.

2.4.3. Cơ sở dữ liệu MySQL

MySQL là hệ quản trị cơ sở dữ liệu quan hệ, sử dụng ngôn ngữ SQL để tạo lập, truy vấn và quản lý dữ liệu. Dữ liệu được tổ chức trong các bảng gồm hàng và cột, giúp việc lưu trữ, tìm kiếm và tổng hợp thông tin được thực hiện có cấu trúc.

Trong hệ thống IoT giám sát kho hàng, MySQL được sử dụng để lưu trữ dữ liệu cảm biến và lịch sử cảnh báo. Các trường dữ liệu giám sát có thể bao gồm thời gian ghi nhận, nhiệt độ, độ ẩm, giá trị rung động và trạng thái cảnh báo tương ứng.

Việc lưu trữ dữ liệu trong cơ sở dữ liệu giúp hệ thống thực hiện các chức năng như:

- Truy xuất các bản ghi cảm biến đã được ghi nhận.
- Hiển thị dữ liệu lịch sử trên dashboard.
- Theo dõi các sự kiện cảnh báo đã xảy ra.
- Hỗ trợ việc kiểm tra và đánh giá quá trình hoạt động của hệ thống.

Backend kết nối đến MySQL thông qua thư viện mysql2 trong môi trường Node.js. Khi nhận được dữ liệu phù hợp, backend thực hiện các thao tác ghi hoặc truy vấn cơ sở dữ liệu theo chương trình đã xây dựng.

Cần phân biệt dữ liệu hiện tại được truyền qua MQTT hoặc WebSocket với dữ liệu lịch sử được truy vấn từ MySQL. Sự kết hợp giữa hai phương thức này giúp giao diện vừa hiển thị thông tin mới vừa hỗ trợ xem lại dữ liệu đã lưu.

2.5. Nguyên lý giám sát và điều khiển tự động

2.5.1. Nguyên lý giám sát theo ngưỡng

Giám sát theo ngưỡng là phương pháp so sánh giá trị đo được với một giới hạn đã được thiết lập trước. Khi giá trị đo vượt quá giới hạn, hệ thống xác định điều kiện cảnh báo và thực hiện hành động tương ứng.

Trong đề tài, các ngưỡng giám sát được cấu hình trong chương trình ESP32 như sau:

Thông số	Ngưỡng cấu hình	Đơn vị
Nhiệt độ	35,0	°C
Độ ẩm	80,0	%
Rung động	1,5	m/s^2

Khi nhiệt độ vượt ngưỡng hoặc độ ẩm vượt ngưỡng, hệ thống kích hoạt quạt thông gió và đèn báo theo logic điều khiển đã lập trình. Khi mức rung động được xử lý vượt ngưỡng, hệ thống kích hoạt còi cảnh báo.

Các ngưỡng trên là giá trị cấu hình của mô hình thử nghiệm, không phải giới hạn bảo quản chung cho mọi loại thực phẩm và nông sản. Khi áp dụng trong thực tế, ngưỡng cần được xác định dựa trên yêu cầu bảo quản của từng loại hàng hóa và kết quả đánh giá hệ thống.

2.5.2. Chế độ điều khiển tự động

Ở chế độ tự động, ESP32 sử dụng dữ liệu cảm biến và các ngưỡng đã thiết lập để xác định trạng thái điều khiển thiết bị. Quá trình này được thực hiện lặp lại trong khi hệ thống hoạt động.

Khi phát hiện nhiệt độ hoặc độ ẩm vượt ngưỡng, hệ thống kích hoạt các thiết bị được quy định trong chương trình. Khi điều kiện không còn vượt ngưỡng, hệ thống cập nhật trạng thái điều khiển theo logic đã lập trình.

Đối với rung động, khi giá trị được sử dụng để giám sát vượt ngưỡng, còi cảnh báo được kích hoạt. Trong mô hình, còi có thể được điều khiển đóng ngắt liên tục để tạo tín hiệu âm thanh cảnh báo.

Chế độ tự động giúp hệ thống phản hồi theo các điều kiện giám sát mà không yêu cầu người dùng trực tiếp thao tác cho từng lần cảnh báo.

2.5.3. Chế độ điều khiển thủ công

Ở chế độ thủ công, người dùng thực hiện thao tác trên dashboard để gửi lệnh điều khiển thiết bị. Lệnh được backend tiếp nhận và chuyển qua MQTT đến ESP32. Sau khi nhận lệnh, ESP32 xử lý và điều khiển relay tương ứng.

Chế độ này cho phép người dùng chủ động bật hoặc tắt quạt, đèn báo và còi cảnh báo phục vụ việc kiểm tra hoặc vận hành mô hình.

Việc phân chia chế độ tự động và thủ công giúp tăng tính linh hoạt trong quá trình vận hành. Tuy nhiên, chương trình cần quản lý rõ trạng thái chế độ để tránh việc lệnh thủ công và logic tự động tác động không nhất quán lên cùng một thiết bị.

CHƯƠNG 3. PHÂN TÍCH VÀ THIẾT HỆ THỐNG

3.1. Phân tích yêu cầu hệ thống

3.1.1. Yêu cầu chức năng

Hệ thống IoT giám sát kho hàng được xây dựng nhằm theo dõi các thông số môi trường, phát hiện điều kiện bất thường và hỗ trợ điều khiển thiết bị. Căn cứ vào mục tiêu của đề tài, hệ thống cần đáp ứng các yêu cầu chức năng sau:

a) Thu thập dữ liệu cảm biến

Hệ thống sử dụng cảm biến DHT22 để thu thập nhiệt độ và độ ẩm môi trường, đồng thời sử dụng cảm biến MPU6050 để thu thập dữ liệu gia tốc phục vụ giám sát rung động. ESP32 tiếp nhận và xử lý dữ liệu từ các cảm biến theo chu kỳ được thiết lập trong chương trình.

b) Hiển thị thông tin giám sát

Các thông số cảm biến cần được hiển thị trên màn hình LCD 16x2 tại mô hình và giao diện dashboard trên máy tính. Giao diện web cần thể hiện được giá trị nhiệt độ, độ ẩm, rung động và trạng thái hoạt động của hệ thống để người dùng dễ dàng theo dõi.

c) Truyền dữ liệu qua mạng

ESP32 sử dụng kết nối Wi-Fi để truyền dữ liệu cảm biến đến hệ thống máy chủ thông qua giao thức MQTT và nền tảng HiveMQ Cloud. Backend tiếp nhận dữ liệu từ broker để phục vụ xử lý, lưu trữ và cập nhật giao diện giám sát.

d) Phát hiện và cảnh báo bất thường

Hệ thống thực hiện so sánh dữ liệu cảm biến với các ngưỡng được thiết lập. Khi nhiệt độ hoặc độ ẩm vượt quá giới hạn, hệ thống kích hoạt quạt thông gió và đèn báo theo logic điều khiển. Khi mức rung động vượt ngưỡng, hệ thống kích hoạt còi cảnh báo.

e) Điều khiển thiết bị

Hệ thống hỗ trợ hai chế độ vận hành:

- Chế độ tự động: ESP32 căn cứ vào dữ liệu cảm biến và điều kiện đã lập trình để điều khiển các thiết bị đầu ra.
- Chế độ thủ công: Người dùng thao tác trên dashboard để gửi lệnh điều khiển thiết bị thông qua backend và MQTT.

f) Lưu trữ và truy xuất dữ liệu

Backend sử dụng cơ sở dữ liệu MySQL để lưu trữ dữ liệu cảm biến và thông tin cảnh báo. Người dùng có thể truy xuất dữ liệu lịch sử thông qua giao diện dashboard nhằm theo dõi diễn biến các thông số trong quá trình vận hành.

3.1.2. Yêu cầu phi chức năng

Bên cạnh các chức năng chính, hệ thống cần đáp ứng một số yêu cầu phi chức năng nhằm bảo đảm khả năng vận hành và thuận tiện trong quá trình sử dụng.

- Tính ổn định: Các thành phần phần cứng và phần mềm cần hoạt động phối hợp, hạn chế lỗi trong quá trình thu thập và truyền dữ liệu.
- Tính trực quan: Giao diện giám sát cần bố trí thông tin rõ ràng, dễ quan sát và thuận tiện cho thao tác điều khiển.
- Khả năng cập nhật: Dữ liệu mới cần được truyền và hiển thị kịp thời để người dùng theo dõi trạng thái hệ thống.
- Khả năng lưu trữ: Dữ liệu cảm biến và thông tin cảnh báo cần được lưu lại để phục vụ việc tra cứu.
- Khả năng mở rộng: Hệ thống cần có cấu trúc cho phép bổ sung cảm biến, thiết bị đầu ra hoặc chức năng phần mềm trong tương lai.

3.1.3. Các thông số và điều kiện giám sát

Trong mô hình thử nghiệm, các ngưỡng giám sát được thiết lập trong chương trình như sau:

Thông số	Ngưỡng cấu hình	Đơn vị	Hành động dự kiến
Nhiệt độ	35,0	°C	Kích hoạt quạt và đèn báo khi vượt ngưỡng
Độ ẩm	80,0	%	Kích hoạt quạt và đèn báo khi vượt ngưỡng
Rung động	1,5	m/s^2	Kích hoạt còi cảnh báo khi vượt ngưỡng

Đối với nhiệt độ và độ ẩm, điều kiện kích hoạt được xác định theo phép toán logic OR. Điều đó có nghĩa là chỉ cần một trong hai thông số vượt ngưỡng thì hệ thống sẽ kích hoạt các thiết bị được quy định.

Đối với rung động, hệ thống sử dụng giá trị rung được tính toán hoặc xử lý từ dữ liệu MPU6050 để so sánh với ngưỡng. Phương pháp tính giá trị rung cần thống nhất với chương trình thực tế của ESP32.

Các ngưỡng trên phục vụ kiểm thử mô hình, không phải tiêu chuẩn bảo quản áp dụng chung cho mọi loại thực phẩm và nông sản.

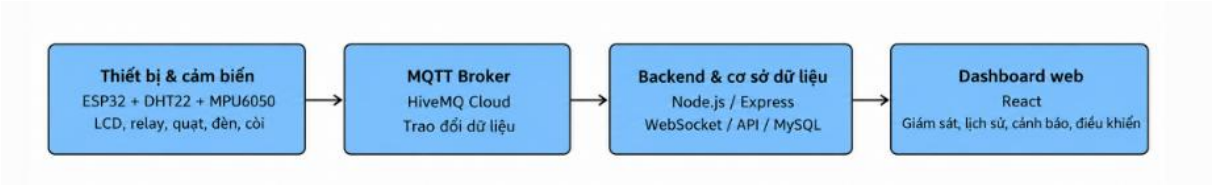
3.2. Thiết kế kiến trúc tổng thể

3.2.1. Mô hình kiến trúc hệ thống

Hệ thống được thiết kế theo mô hình gồm ba nhóm thành phần chính: thiết bị phần cứng tại kho hàng, hệ thống xử lý và lưu trữ dữ liệu, và giao diện giám sát dành cho người dùng.

Nhóm thiết bị phần cứng thực hiện thu thập thông số môi trường và điều khiển các thiết bị đầu ra. Nhóm xử lý dữ liệu tiếp nhận thông tin từ ESP32 thông qua MQTT, lưu trữ dữ liệu trong MySQL và cung cấp thông tin cho giao diện. Nhóm giao diện cho phép người dùng theo dõi dữ liệu, xem lịch sử và gửi lệnh điều khiển.

Kiến trúc tổng thể được mô tả như sau:



Hình 3.1. Sơ đồ kiến trúc tổng thể của hệ thống IoT giám sát kho hàng

Trong kiến trúc trên, ESP32 và backend trao đổi dữ liệu thông qua MQTT Broker. Backend thực hiện kết nối với MySQL để lưu trữ hoặc truy vấn dữ liệu, đồng thời cung cấp thông tin cho dashboard thông qua WebSocket và các API phù hợp.

Đối với chiều điều khiển, người dùng thao tác trên dashboard, backend chuyển tiếp lệnh qua MQTT và ESP32 tiếp nhận lệnh để điều khiển thiết bị trong chế độ thủ công. Ở chế độ tự động, ESP32 căn cứ vào dữ liệu cảm biến và logic được lập trình để điều khiển các thiết bị đầu ra.

3.2.2. Nguyên lý trao đổi dữ liệu giữa các thành phần

Quá trình trao đổi dữ liệu của hệ thống được thực hiện theo các bước sau:

Bước 1: Thu thập dữ liệu. DHT22 đo nhiệt độ và độ ẩm, MPU6050 thu thập dữ liệu gia tốc. ESP32 đọc dữ liệu từ các cảm biến và thực hiện xử lý ban đầu.

Bước 2: Kiểm tra điều kiện giám sát. ESP32 so sánh các giá trị liên quan với ngưỡng đã thiết lập để xác định trạng thái bình thường hoặc trạng thái cảnh báo. Logic điều khiển tự động được thực hiện tại thiết bị theo chương trình.

Bước 3: Truyền dữ liệu lên máy chủ. ESP32 đóng gói dữ liệu thành thông điệp JSON và gửi đến topic MQTT tương ứng trên HiveMQ Cloud.

Bước 4: Tiếp nhận và xử lý dữ liệu. Backend đăng ký nhận thông điệp từ MQTT Broker, phân tích dữ liệu và cập nhật trạng thái giám sát.

Bước 5: Lưu trữ dữ liệu. Backend ghi các thông số cảm biến và thông tin cảnh báo phù hợp vào cơ sở dữ liệu MySQL.

Bước 6: Cập nhật giao diện. Dashboard nhận dữ liệu mới từ backend thông qua WebSocket hoặc truy xuất dữ liệu bằng API để hiển thị các thông số và biểu đồ.

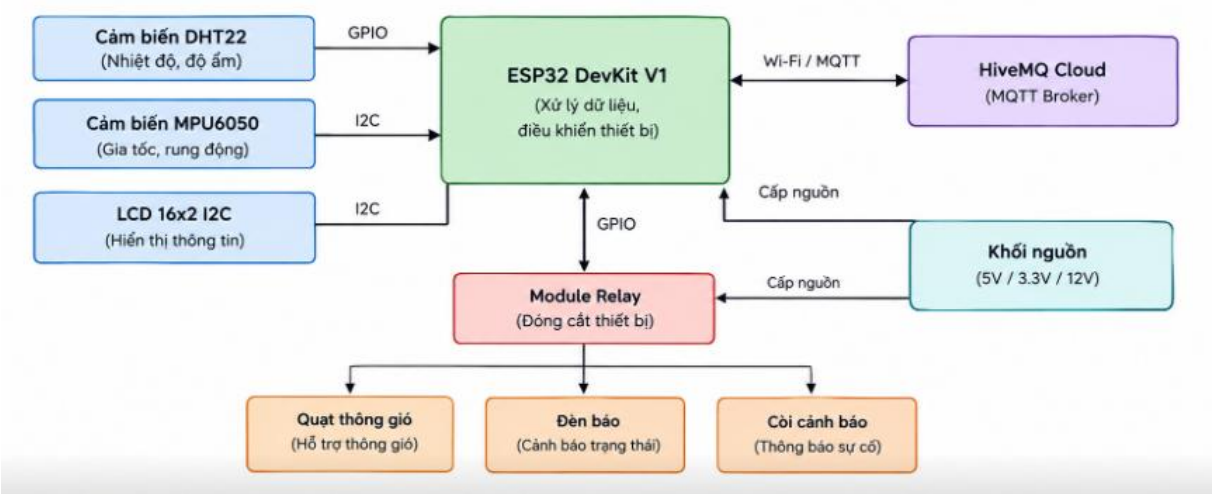
Bước 7: Điều khiển thiết bị. Khi người dùng gửi lệnh trong chế độ thủ công, backend chuyển tiếp lệnh qua MQTT để ESP32 thực hiện. Trạng thái thiết bị có thể được gửi lại để cập nhật giao diện.

Quy trình này tạo thành luồng trao đổi dữ liệu giữa phần cứng và phần mềm, đồng thời phân tách chức năng thu thập dữ liệu, xử lý nghiệp vụ, lưu trữ và hiển thị thông tin.

3.3. Thiết kế phần cứng

3.3.1. Sơ đồ khối phần cứng

Phần cứng của hệ thống gồm khối cảm biến, khối xử lý trung tâm, khối hiển thị, khối điều khiển tải và khối nguồn.



Hình 3.2. Sơ đồ khối phần cứng của hệ thống.

ESP32 là trung tâm xử lý và kết nối các khối phần cứng. Cảm biến cung cấp dữ liệu đầu vào, LCD hiển thị thông tin tại mô hình, còn module relay thực hiện việc đóng cắt các tải đầu ra theo tín hiệu điều khiển.

3.3.2. Thiết kế nối cảm biến và thiết bị

Các kết nối phần cứng được lựa chọn dựa trên giao tiếp của từng linh kiện và khả năng tương thích với ESP32.

Thiết bị	Giao tiếp	Chức năng
DHT22	Tín hiệu số	Đo nhiệt độ và độ ẩm
MPU6050	I2C	Thu thập dữ liệu gia tốc
LCD 16x2 I2C	I2C	Hiển thị thông tin tại mô hình
Module relay	GPIO điều khiển	Đóng cắt các tải đầu ra
Quạt thông gió	Qua tiếp điểm relay	Hỗ trợ thông gió
Đèn báo	Qua tiếp điểm relay	Báo trạng thái hoặc cảnh báo
Còi cảnh báo	Qua tiếp điểm relay	Phát tín hiệu âm thanh

MPU6050 và LCD sử dụng giao tiếp I2C để trao đổi dữ liệu với ESP32. Hai thiết bị có thể dùng chung đường SDA và SCL nếu địa chỉ I2C không xung đột và việc đấu nối đáp ứng yêu cầu điện của các module.

Đối với DHT22, chân dữ liệu được kết nối với chân GPIO được cấu hình trong chương trình. Các chân điều khiển relay cũng được gán theo chương trình ESP32 để quản lý từng thiết bị đầu ra.

Việc lựa chọn chân GPIO cụ thể phải thống nhất giữa sơ đồ đấu dây và mã nguồn đang sử dụng. Khi hoàn thiện báo cáo, cần bổ sung bảng chân kết nối chính xác theo mô hình thực tế, tránh sử dụng các chân tham khảo không trùng với mạch đã lắp ráp.

3.4. Thiết kế phần mềm

3.4.1. Thiết kế chương trình trên ESP32

Chương trình trên ESP32 được xây dựng nhằm quản lý quá trình thu thập dữ liệu, giám sát điều kiện môi trường, truyền thông MQTT và điều khiển thiết bị đầu ra.

Các chức năng chính gồm:

- Khởi tạo giao tiếp với DHT22, MPU6050 và LCD.
- Kết nối mạng Wi-Fi và MQTT Broker.
- Đọc dữ liệu cảm biến theo chu kỳ.
- Xử lý dữ liệu và kiểm tra các điều kiện vượt ngưỡng.
- Điều khiển relay theo chế độ tự động hoặc theo lệnh thủ công.
- Đóng gói và gửi dữ liệu cảm biến qua MQTT.
- Tiếp nhận lệnh điều khiển từ topic MQTT.
- Cập nhật trạng thái hoạt động và xử lý kết nối lại khi cần thiết.

Chương trình được tổ chức theo các chức năng riêng để dễ kiểm tra và điều chỉnh. Việc tách biệt phần đọc cảm biến, xử lý điều kiện, truyền thông và điều khiển thiết bị giúp thuận tiện cho việc phát triển cũng như phát hiện lỗi.

3.4.2. Thiết kế backend

Backend được triển khai bằng Node.js và Express.js, sử dụng thư viện MQTT để kết nối với HiveMQ Cloud, thư viện mysql2 để làm việc với MySQL và WebSocket để trao đổi dữ liệu thời gian thực với dashboard.

Các chức năng chính của backend bao gồm:

- Quản lý kết nối MQTT: Thiết lập kết nối đến HiveMQ Cloud, đăng ký các topic cần thiết và tiếp nhận dữ liệu do ESP32 gửi lên.
- Xử lý dữ liệu cảm biến: Phân tích thông điệp JSON, trích xuất các giá trị cần thiết và cập nhật trạng thái dữ liệu phục vụ dashboard.

- Lưu trữ dữ liệu: Ghi dữ liệu cảm biến và thông tin cảnh báo vào các bảng tương ứng trong MySQL.
- Cung cấp API: Xây dựng các tuyến API phục vụ việc kiểm tra trạng thái backend, truy vấn dữ liệu lịch sử và cung cấp thông tin cho giao diện.
- Trao đổi dữ liệu thời gian thực: Sử dụng WebSocket để gửi dữ liệu mới đến các kết nối dashboard đang hoạt động.

Việc tổ chức các chức năng này trong backend giúp tập trung xử lý dữ liệu và giảm sự phụ thuộc trực tiếp của dashboard vào thiết bị phần cứng.

3.4.3. Thiết kế cơ sở dữ liệu

Cơ sở dữ liệu MySQL được sử dụng để lưu trữ thông tin phục vụ giám sát và truy xuất lịch sử. Dựa trên các chức năng của hệ thống, dữ liệu được tổ chức thành các nhóm chính gồm dữ liệu cảm biến và lịch sử cảnh báo.

Bảng dữ liệu cảm biến:

Bảng này lưu các bản ghi dữ liệu được tiếp nhận trong quá trình vận hành. Các trường dữ liệu có thể bao gồm:

Trường dữ liệu	**Ý nghĩa**
id	Mã định danh bản ghi
recorded_at	Thời điểm ghi nhận
temperature	Nhiệt độ đo được
humidity	Độ ẩm đo được
vibration	Giá trị rung động
temp_alert	Trạng thái cảnh báo nhiệt độ
hum_alert	Trạng thái cảnh báo độ ẩm
vib_alert	Trạng thái cảnh báo rung động

Bảng lịch sử cảnh báo:

Bảng này phục vụ việc lưu trữ và truy xuất các sự kiện cảnh báo trong quá trình hoạt động. Tùy theo cấu trúc triển khai thực tế, dữ liệu có thể bao gồm mã sự kiện, loại cảnh báo, thời điểm phát sinh và trạng thái xử lý.

Việc phân tách dữ liệu cảm biến và thông tin cảnh báo giúp thuận tiện cho việc truy vấn lịch sử, hiển thị danh sách sự kiện và đánh giá trạng thái hệ thống.

Cấu trúc trường cụ thể của bảng alert_history cần được đối chiếu với lược đồ MySQL thực tế trước khi hoàn thiện báo cáo. Bên cạnh đó, các trường và kiểu dữ liệu trong bảng sensor_history cũng cần thống nhất với mã nguồn backend.

3.4.4. Thiết kế giao diện dashboard

Giao diện dashboard được xây dựng bằng React, sử dụng Recharts để hiển thị biểu đồ và Lucide React để cung cấp biểu tượng cho các thành phần giao diện.

Giao diện được tổ chức thành các khu vực chức năng chính:

- Dashboard: Hiển thị các thông số nhiệt độ, độ ẩm, rung động, trạng thái kho hàng và biểu đồ theo dõi.
- History: Cho phép truy xuất và xem lại dữ liệu cảm biến đã lưu trong MySQL.
- Alerts: Hiển thị các sự kiện cảnh báo, hỗ trợ người dùng theo dõi tình trạng bất thường.
- Settings: Cung cấp các chức năng cấu hình được triển khai cho hệ thống, chẳng hạn như lựa chọn chế độ vận hành và điều chỉnh ngưỡng nếu được hỗ trợ trong phiên bản phần mềm.

Ngoài việc hiển thị dữ liệu, dashboard còn cung cấp các thao tác điều khiển thiết bị. Khi người dùng thực hiện thao tác ở chế độ thủ công, giao diện gửi lệnh đến backend để chuyển tiếp đến ESP32 thông qua MQTT.

Thiết kế giao diện hướng đến sự rõ ràng, dễ sử dụng và phù hợp với mục tiêu giám sát. Các thông số quan trọng cần được bố trí ở vị trí dễ quan sát, trong

khi lịch sử và cảnh báo được tổ chức thành các khu vực riêng biệt để người dùng dễ tra cứu.

3.5. Thiết kế thuật toán điều khiển

3.5.1. Thuật toán giám sát nhiệt độ và độ ẩm

Thuật toán giám sát nhiệt độ và độ ẩm được thực hiện bằng cách đọc dữ liệu từ DHT22, sau đó so sánh với các ngưỡng đã cấu hình.

Quy trình xử lý như sau:

1. ESP32 đọc nhiệt độ và độ ẩm từ DHT22.
2. Kiểm tra tính hợp lệ của dữ liệu cảm biến.
3. So sánh nhiệt độ với ngưỡng 35,0 °C.
4. So sánh độ ẩm với ngưỡng 80,0 %RH.
5. Nếu nhiệt độ hoặc độ ẩm vượt ngưỡng, kích hoạt quạt và đèn báo theo logic tự động.
6. Nếu cả hai thông số không còn vượt ngưỡng, cập nhật trạng thái thiết bị theo chương trình.
7. Gửi dữ liệu cảm biến và trạng thái liên quan lên hệ thống giám sát.

Thuật toán có thể biểu diễn bằng mã giả như sau:

Đọc nhiệt độ và độ ẩm

Nếu dữ liệu hợp lệ:

Nếu nhiệt độ > 35 hoặc độ ẩm > 80:

Bật quạt

Bật đèn báo

Ngược lại:

Tắt quạt

Tắt đèn báo

Gửi dữ liệu lên MQTT

Cập nhật trạng thái giám sát

Mã giả trên mô tả nguyên lý ở chế độ tự động. Khi triển khai thực tế, cần xử lý thêm trường hợp dữ liệu cảm biến không hợp lệ và bảo đảm các thao tác điều khiển phù hợp với chế độ vận hành đang được chọn.

3.5.2. Thuật toán cảnh báo rung động

MPU6050 cung cấp dữ liệu gia tốc theo ba trục. Chương trình sử dụng dữ liệu này để tính toán hoặc xác định giá trị phục vụ giám sát rung động, sau đó so sánh với ngưỡng đã thiết lập.

Quy trình xử lý gồm:

1. Đọc dữ liệu gia tốc từ MPU6050.
2. Xử lý dữ liệu theo phương pháp được cài đặt trong chương trình.
3. Xác định giá trị rung động dùng cho việc giám sát.
4. So sánh giá trị với ngưỡng 1,5 m/s^2.
5. Nếu giá trị vượt ngưỡng, kích hoạt còi cảnh báo.
6. Nếu giá trị không còn vượt ngưỡng, cập nhật trạng thái còi theo logic điều khiển.
7. Gửi thông tin rung động và trạng thái cảnh báo đến backend.

Trong trường hợp còi được điều khiển thông qua relay đóng ngắt liên tục, chương trình cần thực hiện chu kỳ bật và tắt phù hợp để tạo tín hiệu âm thanh. Cách xử lý dữ liệu và cách tạo tín hiệu còi cần thống nhất với mã nguồn ESP32 thực tế.

3.5.3. Thuật toán điều khiển theo chế độ vận hành

Hệ thống hỗ trợ hai chế độ điều khiển là tự động và thủ công. Việc lựa chọn chế độ quyết định nguồn lệnh được sử dụng để điều khiển thiết bị.

Ở chế độ tự động, chương trình ESP32 sử dụng dữ liệu cảm biến và các ngưỡng được thiết lập để điều khiển quạt, đèn báo và còi cảnh báo.

Ở chế độ thủ công, ESP32 tiếp nhận lệnh từ backend thông qua MQTT và thực hiện thao tác tương ứng với thiết bị được yêu cầu.

Mã giả tổng quát:

Kiểm tra chế độ vận hành

Nếu chế độ là TỰ ĐỘNG:

- Đọc dữ liệu cảm biến
- Kiểm tra ngưỡng nhiệt độ và độ ẩm
- Kiểm tra ngưỡng rung động
- Điều khiển thiết bị theo điều kiện

Nếu chế độ là THỦ CÔNG:

- Tiếp nhận lệnh điều khiển qua MQTT
- Xác định thiết bị cần điều khiển
- Thực hiện lệnh bật hoặc tắt

Cập nhật trạng thái hệ thống

Gửi dữ liệu giám sát

Việc phân biệt hai chế độ giúp tránh tình trạng các lệnh thủ công và điều kiện tự động tác động không nhất quán lên cùng một thiết bị. Khi chuyển chế độ, chương trình cần cập nhật trạng thái vận hành và thực hiện điều khiển theo quy tắc đã xác định.

3.6. Thiết kế quy trình kiểm thử hệ thống

Để đánh giá khả năng đáp ứng yêu cầu, hệ thống cần được kiểm thử theo từng nhóm chức năng. Quá trình kiểm thử tập trung vào khả năng thu thập dữ liệu, truyền thông, lưu trữ, hiển thị, cảnh báo và điều khiển thiết bị.

Các nội dung kiểm thử dự kiến được trình bày trong bảng sau:

STT	Nội dung kiểm thử	Kết quả cần xác nhận
1	Đọc dữ liệu DHT22	Nhiệt độ và độ ẩm được cập nhật hợp lệ
2	Đọc dữ liệu MPU6050	Giá trị phục vụ giám sát rung động được cập nhật
3	Kết nối MQTT	ESP32 và backend trao đổi được thông điệp
4	Hiển thị dashboard	Các thông số được cập nhật phù hợp với dữ liệu nhận được
5	Lưu trữ MySQL	Dữ liệu cảm biến được ghi và truy vấn thành công
6	Cảnh báo nhiệt độ, độ ẩm	Thiết bị đầu ra phản hồi khi vượt ngưỡng
7	Cảnh báo rung động	Còi hoạt động khi điều kiện rung vượt ngưỡng
8	Điều khiển thủ công	Lệnh từ dashboard điều khiển đúng thiết bị
9	Điều khiển tự động	Thiết bị phản hồi theo các điều kiện đã cấu hình
10	Khôi phục kết nối	Hệ thống có thể tiếp tục hoạt động sau khi kết nối được khôi phục

CHƯƠNG 4. XÂY DỰNG VÀ TRIỂN KHAI HỆ THỐNG

4.1. Xây dựng phần cứng hệ thống

4.1.1. Lắp ráp các thiết bị phần cứng

Hệ thống IoT giám sát kho hàng được xây dựng dựa trên vi điều khiển ESP32 DevKit V1 kết hợp với các cảm biến và thiết bị ngoại vi nhằm thu thập dữ liệu môi trường, phát hiện rung động bất thường và thực hiện các chức năng cảnh báo, điều khiển tự động.

Các linh kiện chính được sử dụng trong hệ thống bao gồm ESP32 DevKit V1, cảm biến nhiệt độ và độ ẩm DHT22, cảm biến gia tốc MPU6050, màn hình LCD 16x2 giao tiếp I2C, module relay, quạt thông gió, đèn báo và còi buzzer.

Quá trình lắp ráp được thực hiện theo từng nhóm chức năng. Nhóm cảm biến có nhiệm vụ thu thập thông tin về môi trường và rung động. Nhóm hiển thị giúp người dùng quan sát thông tin trực tiếp trên mô hình. Nhóm thiết bị chấp hành bao gồm quạt, đèn báo và còi cảnh báo, được điều khiển thông qua module relay.

ESP32 đóng vai trò là bộ điều khiển trung tâm, tiếp nhận dữ liệu từ cảm biến, xử lý các điều kiện giám sát và gửi dữ liệu lên hệ thống thông qua giao thức MQTT. Đồng thời, vi điều khiển điều khiển các thiết bị chấp hành theo chế độ hoạt động được thiết lập.

4.1.2. Kết nối các cảm biến và các thiết bị ngoại vi

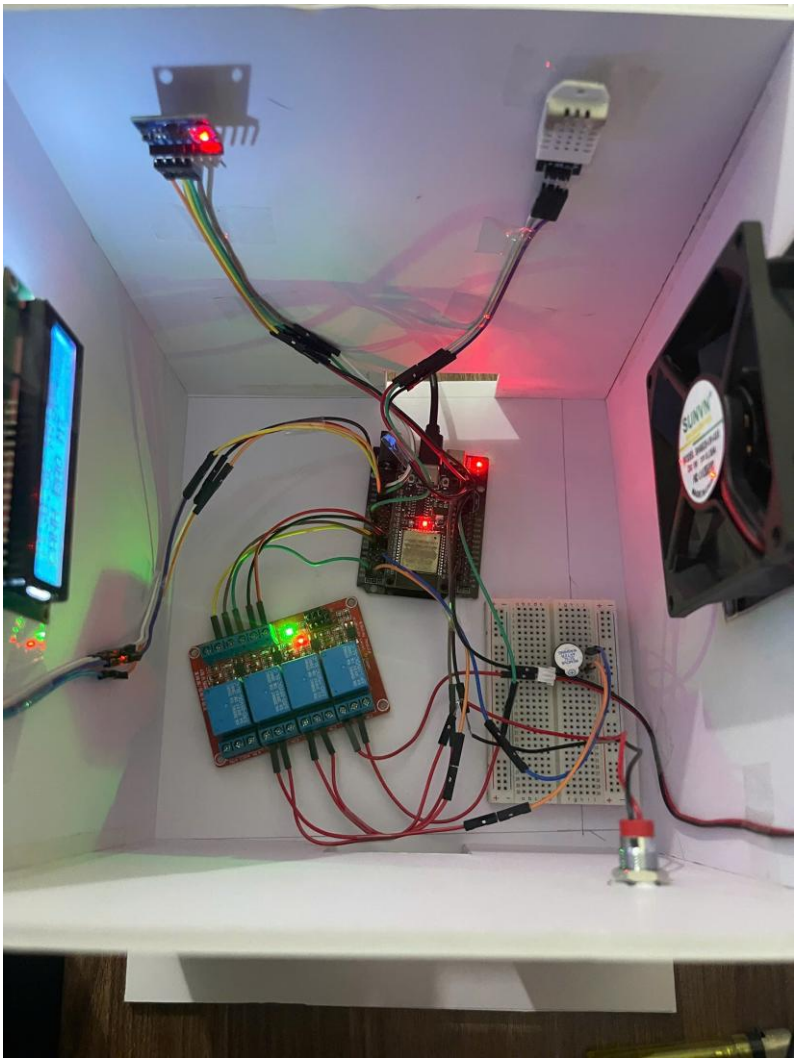
Cảm biến DHT22 được sử dụng để đo nhiệt độ và độ ẩm không khí trong kho. Dữ liệu thu thập được là cơ sở để đánh giá điều kiện môi trường và xác định thời điểm cần kích hoạt quạt thông gió hoặc đèn báo.

Cảm biến MPU6050 được sử dụng để thu thập dữ liệu gia tốc, từ đó hỗ trợ nhận biết các rung động bất thường. Khi giá trị rung động vượt quá ngưỡng cấu hình, hệ thống có thể kích hoạt còi cảnh báo nhằm thông báo cho người vận hành.

Màn hình LCD 16x2 giao tiếp qua chuẩn I2C được tích hợp để hiển thị các thông tin cần thiết như nhiệt độ, độ ẩm hoặc trạng thái hoạt động của hệ thống.

Việc sử dụng giao tiếp I2C giúp giảm số lượng chân kết nối cần thiết giữa màn hình và ESP32.

Module relay được sử dụng làm phần tử đóng cắt các thiết bị ngoại vi. Trong mô hình, relay điều khiển quạt thông gió, đèn báo và còi cảnh báo theo tín hiệu điều khiển từ ESP32. Khi triển khai thực tế, cần bảo đảm nguồn cấp phù hợp với từng thiết bị, sử dụng chung mốc tham chiếu điện áp khi cần thiết và tránh cấp tải vượt quá khả năng của module relay.



Hình 4.1. Mô hình phần cứng hệ thống IoT giám sát kho hàng.

4.2. Xây dựng chương trình cho ESP32

4.2.1. Thiết lập môi trường lập

Chương trình điều khiển được xây dựng cho vi điều khiển ESP32 DevKit V1. Mã nguồn đảm nhiệm các chức năng khởi tạo cảm biến, đọc dữ liệu, xử lý

ngưỡng cảnh báo, điều khiển thiết bị ngoại vi và trao đổi dữ liệu với máy chủ MQTT.

Trong quá trình xây dựng chương trình, các thư viện hỗ trợ được sử dụng để đơn giản hóa việc giao tiếp với cảm biến, kết nối mạng không dây và trao đổi dữ liệu. Việc chia chương trình thành các nhóm chức năng giúp mã nguồn dễ đọc, thuận tiện cho quá trình kiểm thử và mở rộng hệ thống.

Các chức năng chính của chương trình ESP32 gồm:

- Khởi tạo giao tiếp với cảm biến DHT22 và MPU6050.
- Khởi tạo màn hình LCD và các chân điều khiển relay.
- Kết nối Wi-Fi và thiết lập kết nối đến MQTT broker.
- Thu thập, xử lý dữ liệu cảm biến.
- So sánh dữ liệu với các ngưỡng cảnh báo đã cấu hình.
- Điều khiển quạt, đèn báo và còi cảnh báo.
- Gửi dữ liệu giám sát và nhận lệnh điều khiển từ hệ thống.

4.2.2. Thu thập dữ liệu nhiệt độ và độ ẩm

DHT22 được sử dụng để đo nhiệt độ và độ ẩm tại khu vực mô phỏng kho hàng. Sau khi khởi tạo cảm biến, ESP32 thực hiện đọc dữ liệu định kỳ và kiểm tra tính hợp lệ trước khi sử dụng.

Hai thông số được theo dõi là nhiệt độ (T), tính bằng độ C, và độ ẩm tương đối (H), tính bằng phần trăm. Dữ liệu sau khi đọc được sử dụng để cập nhật màn hình hiển thị, gửi lên máy chủ và phục vụ quá trình đánh giá trạng thái môi trường.

Trong chương trình, ngưỡng nhiệt độ được thiết lập ban đầu là 35°C và ngưỡng độ ẩm là 80% RH. Đây là các giá trị cấu hình phục vụ thử nghiệm mô hình, không phải tiêu chuẩn bảo quản chung cho mọi loại thực phẩm và nông sản. Điều kiện cảnh báo môi trường được xác định như sau:

- Nếu nhiệt độ vượt quá ngưỡng cho phép, hệ thống kích hoạt trạng thái cảnh báo nhiệt độ.
- Nếu độ ẩm vượt quá ngưỡng cho phép, hệ thống kích hoạt trạng thái cảnh báo độ ẩm.

- Nếu một trong hai điều kiện trên xảy ra trong chế độ tự động, hệ thống điều khiển quạt thông gió và đèn báo theo logic đã thiết kế.

Việc kết hợp hai thông số giúp hệ thống theo dõi đồng thời các yếu tố môi trường có thể ảnh hưởng đến điều kiện bảo quản hàng hóa.

4.2.3. Thu thập và xử lý dữ liệu rung

MPU6050 là cảm biến tích hợp gia tốc kế và con quay hồi chuyển. Trong phạm vi đề tài, dữ liệu gia tốc được sử dụng để đánh giá sự thay đổi chuyển động của mô hình kho hàng.

ESP32 đọc dữ liệu từ MPU6050 thông qua giao tiếp I2C. Các giá trị thu thập được đưa vào bước xử lý để xác định mức độ rung động. Kết quả được so sánh với ngưỡng rung động được cấu hình trong chương trình, ban đầu là 1,5 m/s^2 theo cách biểu diễn của mô hình.

Khi giá trị rung động vượt quá ngưỡng, hệ thống xác định trạng thái rung động bất thường và kích hoạt còi cảnh báo trong chế độ tự động. Việc điều khiển còi được thực hiện theo cơ chế bật, tắt để tạo tín hiệu cảnh báo.

Cần lưu ý rằng độ chính xác của việc đánh giá rung động phụ thuộc vào cách tính toán giá trị rung, hướng lắp cảm biến, độ nhiễu và phương pháp hiệu chuẩn. Vì vậy, ngưỡng sử dụng trong mô hình cần được kiểm tra bằng thực nghiệm trước khi áp dụng cho môi trường thực tế.

4.2.4. Điều khiển theo chế độ hoạt động

Hệ thống hỗ trợ hai chế độ điều khiển là chế độ tự động và chế độ thủ công.

- Chế độ tự động: ESP32 liên tục theo dõi dữ liệu cảm biến và so sánh với các ngưỡng đã thiết lập. Khi nhiệt độ hoặc độ ẩm vượt ngưỡng, hệ thống điều khiển quạt thông gió và đèn báo. Khi phát hiện rung động vượt ngưỡng, hệ thống kích hoạt còi cảnh báo. Khi điều kiện trở về mức bình thường, trạng thái điều khiển được cập nhật theo logic của chương trình.
- Chế độ thủ công: Người dùng gửi lệnh từ giao diện dashboard để bật hoặc tắt từng thiết bị. Lệnh được truyền đến backend, sau đó được gửi qua MQTT đến ESP32 để thực hiện điều khiển.

Việc phân chia hai chế độ cho phép hệ thống vừa tự động phản ứng trước các điều kiện bất thường, vừa hỗ trợ người dùng chủ động kiểm tra và điều khiển thiết bị khi cần thiết.



Hình 4.2. Lưu đồ xử lý dữ liệu và điều khiển thiết bị trên ESP32.

4.3. Xây dựng hệ thống truyền thông MQTT

4.3.1. Xây dựng các kênh truyền dữ liệu

Trong hệ thống, các topic MQTT được tổ chức theo chức năng để phân biệt dữ liệu cảm biến, lệnh điều khiển và trạng thái thiết bị.

Topic MQTT	Chức năng
iot/sensor/data	Truyền dữ liệu cảm biến từ ESP32
iot/device/control	Truyền lệnh điều khiển từ backend đến ESP32
iot/device/status	Truyền trạng thái thiết bị hoặc hệ thống

Việc phân chia topic giúp các thành phần xác định đúng loại thông tin cần gửi và nhận, đồng thời tạo điều kiện mở rộng hệ thống trong tương lai.

Dữ liệu cảm biến được đóng gói dưới dạng JSON trước khi gửi. Cách biểu diễn này giúp dữ liệu có cấu trúc rõ ràng, thuận tiện cho backend phân tích và chuyển tiếp đến dashboard.

Ví dụ minh họa về cấu trúc dữ liệu JSON:

```
{
 "temperature": 32.0,
 "humidity": 74.0,
 "vibration": 0.6,
 "tempAlert": false,
```

"humAlert": false,
"vibAlert": false
}

Các giá trị trong ví dụ chỉ mang tính minh họa. Cấu trúc và tên trường trong báo cáo cuối cùng cần thống nhất với dữ liệu thực tế được ESP32 gửi lên broker.

4.3.2. Kiểm tra quá trình truyền nhận dữ liệu

Sau khi hoàn thành cấu hình, quá trình truyền nhận được kiểm tra thông qua MQTT Web Client của HiveMQ Cloud. Công cụ này cho phép theo dõi trực tiếp các bản tin được gửi lên topic và kiểm tra nội dung dữ liệu.

Khi ESP32 hoạt động, các bản tin chứa dữ liệu cảm biến được gửi lên topic iot/sensor/data. Người kiểm thử có thể quan sát nhiệt độ, độ ẩm và giá trị rung động, đồng thời kiểm tra trạng thái cảnh báo tương ứng.

Tiếp theo, lệnh điều khiển được gửi từ backend thông qua topic iot/device/control. ESP32 nhận lệnh và cập nhật trạng thái thiết bị theo nội dung yêu cầu.

Quá trình kiểm tra này giúp xác nhận khả năng trao đổi dữ liệu giữa ESP32 và MQTT broker trước khi tích hợp hoàn chỉnh với backend và dashboard.

4.4. Xây dựng backend xử lý dữ liệu

4.4.1. Thiết lập môi trường backend

Backend được xây dựng bằng Node.js, sử dụng Express để cung cấp các API phục vụ giao tiếp với dashboard. Bên cạnh đó, thư viện MQTT được sử dụng để kết nối với HiveMQ Cloud, thư viện WebSocket hỗ trợ truyền dữ liệu theo thời gian thực và thư viện mysql2 được sử dụng để kết nối cơ sở dữ liệu MySQL.

Mã nguồn backend được tổ chức trong thư mục backend của dự án. Các thành phần chính bao gồm tệp cấu hình môi trường, module kết nối MQTT và chương trình máy chủ.

Các thông tin như địa chỉ máy chủ MQTT, tài khoản kết nối và thông tin cơ sở dữ liệu được cấu hình thông qua biến môi trường. Cách tổ chức này giúp

tách thông tin cấu hình khỏi mã nguồn và thuận tiện hơn khi thay đổi môi trường triển khai.

Khi khởi động, backend thực hiện các công việc cần thiết như khởi tạo kết nối cơ sở dữ liệu, kết nối MQTT, đăng ký nhận dữ liệu và mở máy chủ phục vụ dashboard.

4.4.2. Tiếp nhận và xử lý dữ liệu MQTT

Backend đăng ký theo dõi các topic dữ liệu cảm biến và trạng thái thiết bị trên HiveMQ Cloud. Khi nhận được bản tin, chương trình phân tích nội dung JSON để lấy các thông số cần thiết.

Dữ liệu sau khi phân tích được sử dụng cho các mục đích chính:

- Cập nhật thông tin phục vụ dashboard.
- Ghi nhận dữ liệu cảm biến vào cơ sở dữ liệu.
- Theo dõi trạng thái cảnh báo của hệ thống.
- Cung cấp thông tin cho các API truy vấn lịch sử.

Quá trình xử lý cần kiểm tra định dạng bản tin trước khi sử dụng để hạn chế lỗi khi dữ liệu không hợp lệ hoặc kết nối bị gián đoạn. Đồng thời, việc tách chức năng nhận dữ liệu, lưu trữ và truyền dữ liệu giúp backend dễ bảo trì hơn.

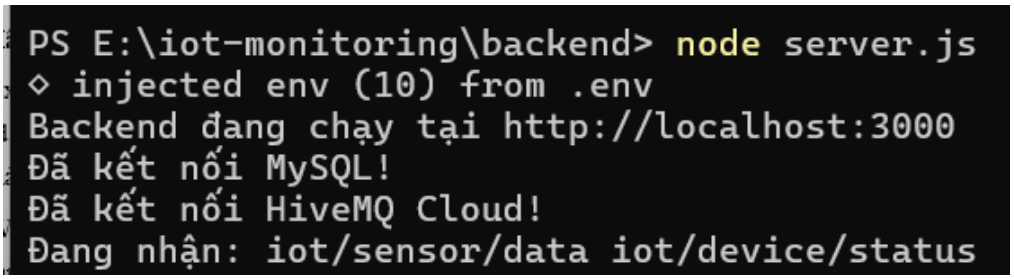
4.4.3. Xây dựng API và truyền dữ liệu thời gian thực

Express được sử dụng để xây dựng các endpoint phục vụ kiểm tra trạng thái backend và truy vấn dữ liệu. Trong đó, endpoint /api/health hỗ trợ kiểm tra khả năng phản hồi của máy chủ.

WebSocket được sử dụng để hỗ trợ trao đổi dữ liệu giữa backend và dashboard mà không cần liên tục tạo các yêu cầu HTTP mới cho từng lần cập nhật. Khi backend nhận được dữ liệu cảm biến, thông tin phù hợp có thể được gửi đến các trình duyệt đang kết nối.

Cơ chế này giúp giao diện hiển thị dữ liệu mới nhanh chóng hơn, hỗ trợ người dùng quan sát sự thay đổi của môi trường kho hàng trong quá trình hệ thống vận hành.

Bên cạnh đó, backend tiếp nhận yêu cầu điều khiển từ dashboard và gửi lệnh qua MQTT. Việc phân chia trách nhiệm giữa REST API, WebSocket và MQTT giúp mỗi giao thức đảm nhiệm một vai trò cụ thể trong hệ thống.



Hình 4.3. Kết quả khởi động backend và kết nối với hệ thống.

4.5. Xây dựng cơ sở dữ liệu MySQL

4.5.1. Thiết lập cơ sở dữ liệu

MySQL được sử dụng để lưu trữ dữ liệu cảm biến phục vụ việc tra cứu lịch sử và đánh giá hoạt động của hệ thống. Cơ sở dữ liệu được kết nối với backend thông qua thư viện mysql2.

Khác với việc chỉ hiển thị dữ liệu hiện tại trên dashboard, cơ sở dữ liệu cho phép lưu lại các bản ghi theo thời gian. Nhờ đó, người dùng có thể xem lại diễn biến nhiệt độ, độ ẩm và rung động trong những khoảng thời gian đã ghi nhận.

Sau khi cấu hình kết nối, backend thực hiện kiểm tra khả năng truy cập cơ sở dữ liệu khi khởi động. Kết quả kết nối thành công cho thấy backend có thể làm việc với MySQL để phục vụ các chức năng lưu trữ và truy vấn.

4.5.2. Lưu trữ lịch sử dữ liệu cảm biến

Bảng sensor_history được sử dụng để lưu các bản ghi dữ liệu cảm biến. Mỗi bản ghi gồm thông tin định danh, thời điểm ghi nhận, nhiệt độ, độ ẩm, giá trị rung động và các cờ cảnh báo tương ứng.

Trường dữ liệu	Ý nghĩa
id	Mã định danh bản ghi
recorded_at	Thời điểm ghi nhận dữ liệu
temperature	Giá trị nhiệt độ
humidity	Giá trị độ ẩm
vibration	Giá trị rung động
temp_alert	Trạng thái cảnh báo nhiệt độ
hum_alert	Trạng thái cảnh báo độ ẩm
vib_alert	Trạng thái cảnh báo rung động

Bảng trên mô tả các trường dữ liệu đã được sử dụng trong kết quả truy vấn lịch sử của hệ thống. Kiểu dữ liệu, ràng buộc và giá trị mặc định cần được trình bày theo cấu trúc bảng thực tế trong MySQL.

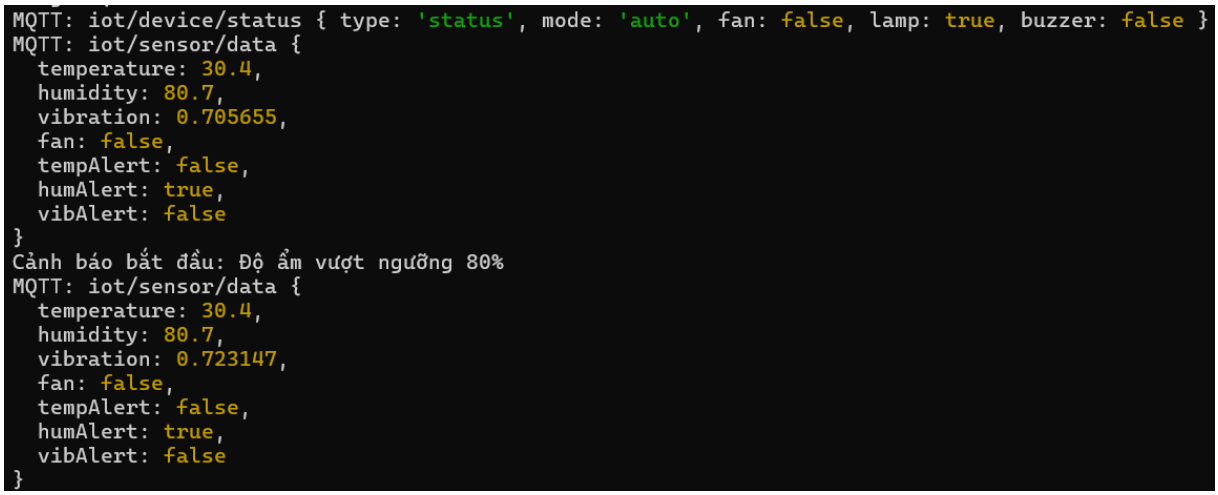
Quá trình ghi dữ liệu được thực hiện từ backend khi tiếp nhận các bản tin cảm biến. Các bản ghi được lưu cùng thời điểm ghi nhận để hỗ trợ sắp xếp, truy vấn và hiển thị lịch sử theo thời gian.

4.5.3. Truy vấn dữ liệu lịch sử

Backend cung cấp API truy vấn lịch sử dữ liệu để dashboard có thể lấy các bản ghi đã lưu trong MySQL. Kết quả trả về được biểu diễn dưới dạng JSON, bao gồm trạng thái truy vấn, số lượng bản ghi và danh sách dữ liệu.

Trong quá trình kiểm thử, API lịch sử đã trả về dữ liệu với cấu trúc gồm các trường thông tin cảm biến và thời gian ghi nhận. Đây là cơ sở để tích hợp chức năng History trên giao diện người dùng.

Việc lưu trữ dữ liệu không chỉ phục vụ hiển thị lịch sử mà còn tạo điều kiện để bổ sung chức năng thống kê, so sánh dữ liệu giữa các khoảng thời gian và đánh giá tần suất xuất hiện của các trạng thái bất thường.



Hình 4.4. Kết quả truy vấn lịch sử dữ liệu cảm biến từ API backend.

4.6. Xây dựng giao diện giám sát và điều khiển

4.6.1. Thiết lập môi trường giao diện

Giao diện người dùng được xây dựng bằng React và Vite. React hỗ trợ tổ chức giao diện thành các thành phần độc lập, trong khi Vite hỗ trợ quá trình phát triển và chạy thử ứng dụng.

Các thư viện Recharts và Lucide React được sử dụng để xây dựng biểu đồ dữ liệu và các biểu tượng trực quan. Giao diện được thiết kế theo phong cách sáng, hướng đến việc trình bày thông tin rõ ràng, dễ quan sát và thuận tiện khi vận hành.

Các chức năng chính được tổ chức thành những khu vực gồm Dashboard, History, Alerts và Settings. Việc phân chia theo chức năng giúp người dùng dễ dàng chuyển đổi giữa màn hình giám sát hiện tại, lịch sử dữ liệu và các nội dung quản lý liên quan.

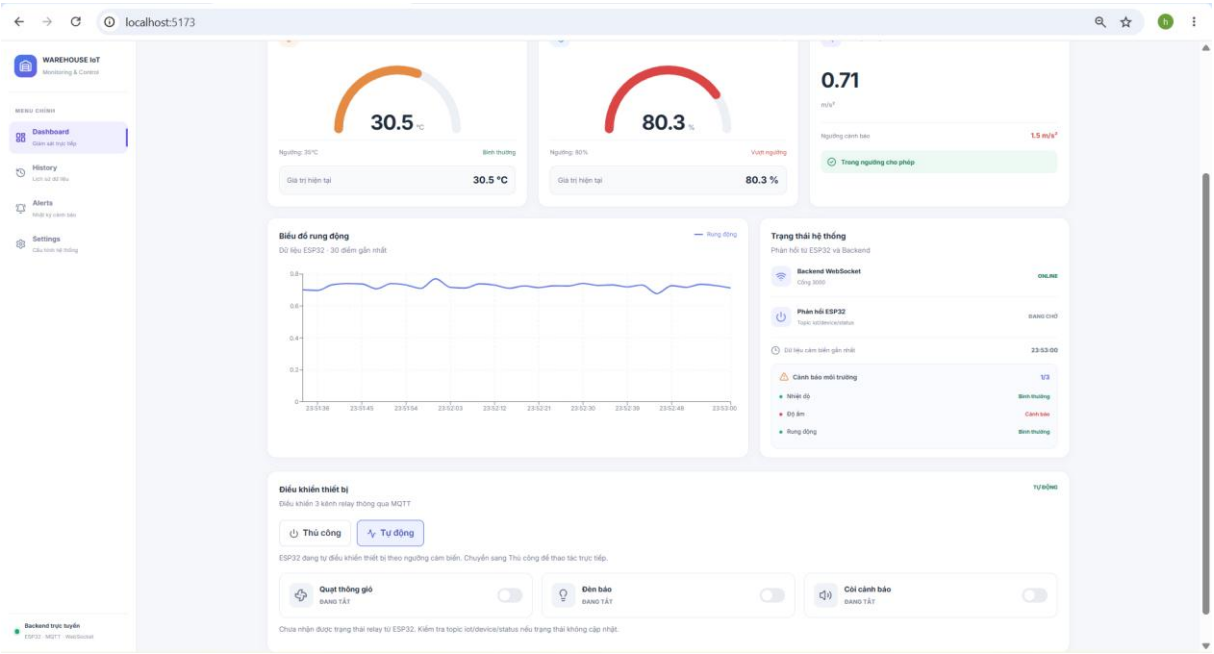
4.6.2. Xây dựng màn hình Dashboard

Dashboard là màn hình chính của hệ thống, tập trung hiển thị tình trạng môi trường kho hàng và trạng thái hoạt động của thiết bị.

Các thẻ thông tin chính bao gồm nhiệt độ, độ ẩm, rung động và trạng thái kho hàng. Các giá trị được cập nhật từ dữ liệu do backend cung cấp, giúp người dùng nhanh chóng nhận biết tình trạng hiện tại.

Biểu đồ được sử dụng để thể hiện sự thay đổi của các thông số theo thời gian. Việc trực quan hóa dữ liệu giúp người vận hành quan sát xu hướng biến đổi của môi trường và phát hiện những thời điểm có thông số bất thường.

Ngoài ra, giao diện hiển thị trạng thái hoạt động của hệ thống và thông tin kết nối khi cần thiết. Điều này giúp người dùng phân biệt trường hợp dữ liệu thực sự ổn định với trường hợp hệ thống đang gặp vấn đề về kết nối.



Hình 4.5. Giao diện Dashboard giám sát nhiệt độ, độ ẩm và rung động.

4.6.3. Xây dựng chức năng điều khiển thiết bị

Giao diện điều khiển cho phép người dùng lựa chọn giữa chế độ tự động và chế độ thủ công.

Ở chế độ tự động, hệ thống thực hiện điều khiển theo các điều kiện được thiết lập trong chương trình ESP32. Dashboard đóng vai trò theo dõi trạng thái và hiển thị thông tin liên quan.

Ở chế độ thủ công, người dùng có thể gửi lệnh bật hoặc tắt quạt thông gió, đèn báo và còi cảnh báo. Sau khi người dùng thực hiện thao tác, lệnh được gửi đến backend, chuyển tiếp qua MQTT và được ESP32 xử lý.

Việc tổ chức chức năng điều khiển trên dashboard giúp giảm sự phụ thuộc vào thao tác trực tiếp trên mô hình, đồng thời cho phép kiểm tra hoạt động của từng thiết bị trong quá trình thử nghiệm.

Cần bảo đảm giao diện thể hiện đúng trạng thái thiết bị đã được xác nhận, tránh chỉ thay đổi biểu tượng trên màn hình mà không kiểm tra kết quả thực hiện lệnh từ ESP32.

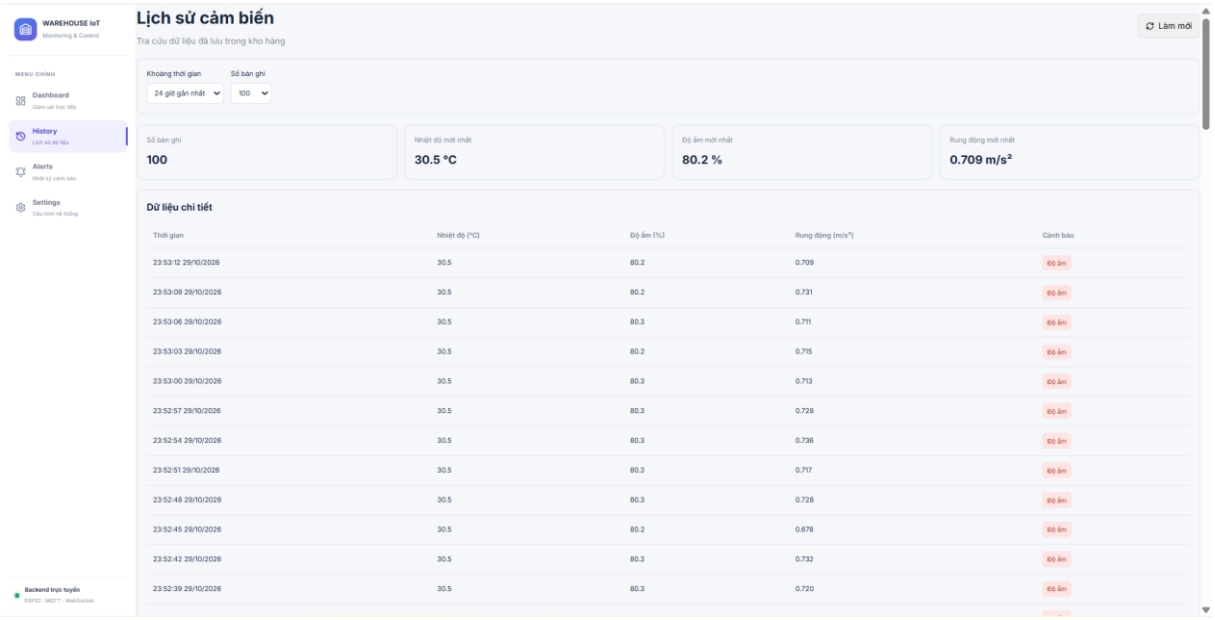
4.6.4. Xây dựng màn hình lịch sử và cảnh báo

Màn hình History được xây dựng để hiển thị các bản ghi cảm biến đã được lưu trong cơ sở dữ liệu. Thông tin có thể bao gồm thời điểm ghi nhận, nhiệt độ, độ ẩm, rung động và trạng thái cảnh báo.

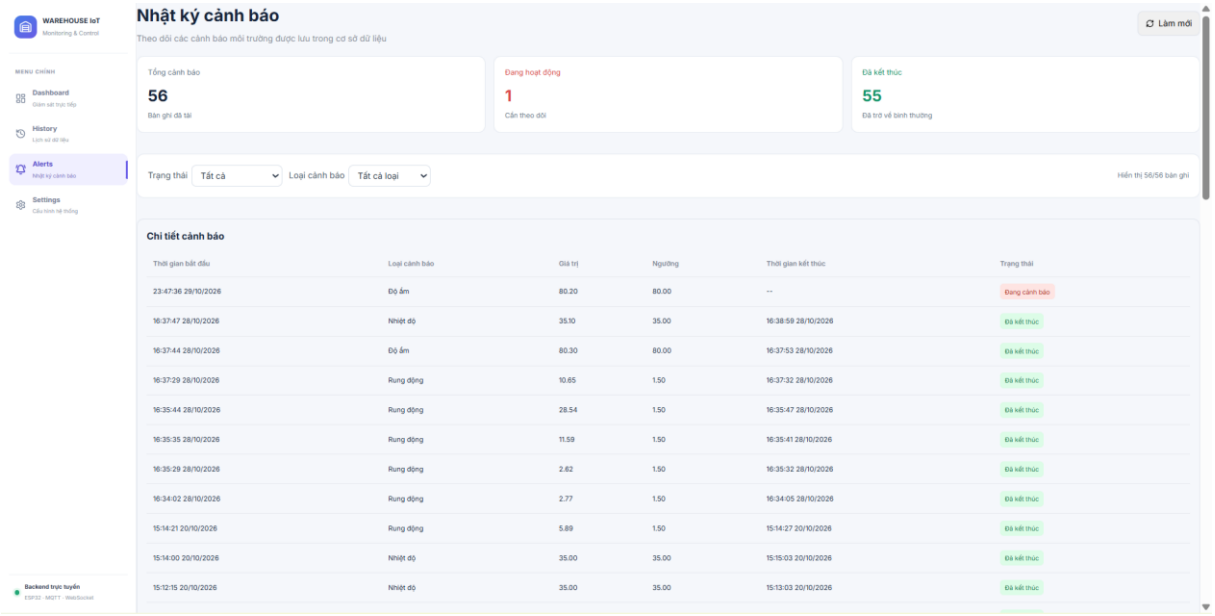
Chức năng này hỗ trợ người dùng xem lại diễn biến của môi trường kho hàng thay vì chỉ quan sát giá trị tức thời. Dữ liệu lịch sử cũng là cơ sở để phân tích các thay đổi bất thường trong quá trình vận hành.

Màn hình Alerts hỗ trợ theo dõi các trạng thái cảnh báo được hệ thống ghi nhận. Việc phân biệt cảnh báo đang xảy ra với cảnh báo đã kết thúc giúp người vận hành nắm bắt tình trạng hiện tại và xem lại những sự kiện đã được ghi nhận.

Đối với các chức năng quản lý ngưỡng trong Settings, nội dung báo cáo cần được cập nhật theo đúng phiên bản dashboard cuối cùng. Nếu giao diện đã cho phép thay đổi ngưỡng và truyền cấu hình đến ESP32, cần mô tả đầy đủ luồng cập nhật. Nếu chức năng này mới dừng ở giao diện hoặc chưa được kiểm thử, không nên khẳng định việc thay đổi ngưỡng đã được áp dụng thành công lên thiết bị.



Hình 4.6. Giao diện lịch sử dữ liệu cảm biến.



Hình 4.7. Giao diện theo dõi cảnh báo của hệ thống.

4.7. Kiểm thử và đánh giá kết quả

4.7.1. Mục tiêu kiểm thử

Kiểm thử được thực hiện nhằm đánh giá khả năng hoạt động của các thành phần phần cứng, phần mềm và khả năng phối hợp giữa chúng. Nội dung kiểm thử tập trung vào việc thu thập dữ liệu, truyền nhận MQTT, lưu trữ lịch sử, hiển thị dashboard, điều khiển thiết bị và phản ứng khi điều kiện môi trường vượt ngưỡng.

4.7.2. Kết quả kiểm thử chức năng

Bảng dưới đây tổng hợp các nội dung đã được thử nghiệm trong quá trình phát triển mô hình.

STT	Nội dung kiểm thử	Kết quả ghi nhận
1	Đọc dữ liệu nhiệt độ và độ ẩm từ DHT22	Đã kiểm tra dữ liệu cảm biến
2	Thu thập và truyền dữ liệu rung động từ MPU6050	Đã kiểm tra dữ liệu trên MQTT
3	Gửi dữ liệu từ ESP32 lên HiveMQ Cloud	Thành công
4	Backend kết nối MQTT và nhận dữ liệu	Thành công
5	Backend kết nối cơ sở dữ liệu MySQL	Thành công
6	Truy vấn dữ liệu lịch sử qua API	Thành công
7	Điều khiển quạt, đèn và còi ở chế độ thủ công	Đã kiểm thử
8	Kích hoạt thiết bị khi điều kiện vượt ngưỡng ở chế độ tự động	Đã kiểm thử
9	Cập nhật trạng thái khi thông số trở về dưới ngưỡng	Đã kiểm thử
10	Khả năng kết nối lại Wi-Fi của ESP32	Đã kiểm thử
11	Hiển thị cảnh báo khi backend mất kết nối	Đã kiểm thử
12	Đối chiếu dữ liệu cảm biến với biểu đồ trên dashboard	Đã kiểm tra
13	Theo dõi hệ thống liên tục trong khoảng 60 phút	Đã thử nghiệm

Kết quả trên cho thấy các chức năng cốt lõi của mô hình đã được kiểm tra trong quá trình phát triển. Tuy nhiên, bảng này phản ánh kết quả kiểm thử chức năng, không thay thế cho việc đánh giá độ chính xác của cảm biến trong điều kiện chuẩn hoặc thử nghiệm dài hạn tại kho hàng thực tế.

4.7.3. Kiểm thử chức năng cảnh báo môi trường

Đối với cảnh báo nhiệt độ và độ ẩm, dữ liệu cảm biến được theo dõi và so sánh với các ngưỡng cấu hình. Khi điều kiện cảnh báo xuất hiện, hệ thống chuyển sang trạng thái tương ứng và thực hiện điều khiển thiết bị theo chế độ tự động.

Sau đó, khi thông số trở về dưới ngưỡng, trạng thái điều khiển được kiểm tra để bảo đảm hệ thống có thể trở lại trạng thái bình thường theo logic đã thiết kế.

Đối với cảnh báo rung động, nhóm tiến hành theo dõi dữ liệu MPU6050 và kiểm tra phản ứng của còi khi giá trị rung động vượt ngưỡng. Bài kiểm thử này nhằm xác nhận sự phối hợp giữa cảm biến, chương trình xử lý và thiết bị cảnh báo.

Kết quả kiểm thử cho thấy cơ chế điều khiển tự động theo ngưỡng và chức năng điều khiển thủ công đã được thử nghiệm. Để đánh giá sâu hơn, có thể bổ sung bảng ghi nhận các giá trị thực tế trước, trong và sau thời điểm cảnh báo.

4.7.4. Kiểm thử truyền dữ liệu và lưu trữ

Quá trình kiểm thử truyền dữ liệu được thực hiện bằng cách quan sát bản tin tại HiveMQ Cloud và đối chiếu với dữ liệu hiển thị trên dashboard. Việc kiểm tra giúp xác nhận dữ liệu đi qua các thành phần của hệ thống và được cập nhật lên giao diện.

Đối với cơ sở dữ liệu, API lịch sử được sử dụng để truy vấn các bản ghi đã lưu. Kết quả trả về có số lượng bản ghi cùng các trường dữ liệu cảm biến và thời điểm ghi nhận. Điều này cho thấy chức năng lưu trữ và truy vấn lịch sử đã được triển khai ở backend.

Ngoài ra, hệ thống đã được kiểm tra trong tình huống mất kết nối backend. Dashboard có thể hiển thị thông tin cảnh báo kết nối, hỗ trợ người dùng nhận biết tình trạng gián đoạn thay vì tiếp tục coi dữ liệu cũ là dữ liệu mới nhất.

4.7.5. Đánh giá ưu điểm và hạn chế

Ưu điểm:

- Hệ thống kết hợp nhiều thành phần phần cứng và phần mềm trong một mô hình giám sát thống nhất.
- Dữ liệu cảm biến có thể được truyền qua MQTT đến backend để xử lý và lưu trữ.
- Dashboard hỗ trợ quan sát thông số môi trường, biểu đồ và lịch sử dữ liệu.
- Người dùng có thể điều khiển thiết bị bằng chế độ thủ công, trong khi chế độ tự động hỗ trợ phản ứng theo các ngưỡng cấu hình.
- Cấu trúc ESP32, MQTT, backend và MySQL tạo điều kiện để bổ sung chức năng trong các giai đoạn phát triển tiếp theo.

Hạn chế:

- Giá trị đo của cảm biến phụ thuộc vào điều kiện lắp đặt, môi trường và quá trình hiệu chuẩn.
- Ngưỡng nhiệt độ, độ ẩm và rung động hiện được sử dụng cho mục đích thử nghiệm, cần được điều chỉnh theo yêu cầu bảo quản cụ thể.
- Hệ thống phụ thuộc vào nguồn điện và kết nối mạng khi sử dụng các chức năng truyền dữ liệu qua MQTT và dashboard.
- Mô hình hiện tại chưa thay thế được các hệ thống giám sát kho hàng chuyên dụng đã được kiểm định trong môi trường vận hành thực tế.
- Cần tiếp tục đánh giá độ ổn định trong thời gian dài, khả năng phục hồi sau sự cố và độ tin cậy của cơ chế cảnh báo.

CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1. Kết luận

Đề tài "Hệ thống IoT giám sát và cảnh báo điều kiện bảo quản thực phẩm" đã xây dựng được mô hình giám sát nhiệt độ, độ ẩm và rung động sử dụng ESP32, cảm biến DHT22 và MPU6050. Dữ liệu được truyền qua MQTT với HiveMQ Cloud, xử lý tại backend Node.js/Express và lưu trữ bằng MySQL.

Bên cạnh đó, hệ thống có giao diện web React giúp theo dõi thông số cảm biến, xem lịch sử dữ liệu, trạng thái cảnh báo và điều khiển thiết bị. Các chức năng chính như truyền dữ liệu, lưu trữ, điều khiển thủ công và tự động theo ngưỡng đã được kiểm thử.

Nhìn chung, đề tài đã đạt được các mục tiêu cơ bản trong phạm vi mô hình thử nghiệm, đồng thời giúp vận dụng kiến thức về IoT, hệ thống nhúng, truyền thông và phát triển phần mềm.

5.2. Hạn chế

Bên cạnh những kết quả đạt được, hệ thống vẫn còn một số hạn chế:

- Độ chính xác của dữ liệu phụ thuộc vào cảm biến và điều kiện môi trường.
- Ngưỡng cảnh báo hiện tại chủ yếu phục vụ thử nghiệm, chưa được đánh giá cho từng loại hàng hóa cụ thể.
- Hệ thống phụ thuộc vào nguồn điện và kết nối mạng khi truyền dữ liệu lên máy chủ.
- Thời gian kiểm thử còn hạn chế, chưa đánh giá đầy đủ khả năng hoạt động lâu dài trong kho hàng thực tế.

5.3. Hướng phát triển

Trong tương lai, hệ thống có thể được cải tiến theo các hướng sau:

- Sử dụng cảm biến có độ chính xác cao hơn và bổ sung chức năng hiệu chuẩn.
- Hoàn thiện chức năng cấu hình ngưỡng cảnh báo trên dashboard và đồng bộ với ESP32.

- Bổ sung thống kê, lọc dữ liệu lịch sử và các hình thức cảnh báo từ xa.
- Ứng dụng trí tuệ nhân tạo để phân tích dữ liệu, phát hiện bất thường và dự báo nguy cơ ảnh hưởng đến hàng hóa.
- Mở rộng hệ thống để giám sát nhiều khu vực kho hàng và nâng cao khả năng vận hành ổn định.

5.4. Tổng kết

Đề tài đã xây dựng được mô hình IoT kết hợp phần cứng, truyền thông MQTT, backend, cơ sở dữ liệu và giao diện web nhằm phục vụ giám sát kho hàng. Kết quả thực hiện là nền tảng để tiếp tục hoàn thiện hệ thống theo hướng chính xác, ổn định và có tính ứng dụng cao hơn trong thực tế.

.

TÀI LIỆU THAM KHẢO

[1] Espressif Systems, *ESP32 Series Datasheet*. Available: https://documentation.espressif.com/esp32_datasheet_en.html

[2] Aosong Electronics, *AM2302/DHT22 Temperature and Humidity Sensor Datasheet*.

[3] TDK InvenSense, *MPU-6000 and MPU-6050 Product Specification*.

[4] OASIS, *MQTT Version 3.1.1*. Available: https://docs.oasis-open.org/mqtt/mqtt/v3.1.1/

[5] HiveMQ, *HiveMQ Cloud Documentation*. Available: https://docs.hivemq.com/

[6] OpenJS Foundation, *Node.js Documentation*. Available: https://nodejs.org/docs/

[7] Express.js, *Express Documentation*. Available: https://expressjs.com/

[8] Oracle, *MySQL 8.0 Reference Manual*. Available: https://dev.mysql.com/doc/refman/8.0/en/

[9] Meta Open Source, *React Documentation*. Available: https://react.dev/

[10] Recharts, *Recharts Documentation*. Available: https://recharts.org/